Intro to CS

67101-2

@einavye , 2024B , טל"ח

TOC

Lecture 1: Introduction, functions, variables, types, expressions 3

Basic data types (values) in python *3*

פעולות חשבון בפייתון *4*

Strings *4*

Behaviour in the shell *5*

Functions *5*

נוסף *7*

Recitation 1 7

Python Turtle *7*

Lecture 2: Conditionals, list, strings, slicing, functions 7

ASCII and UNICODE *7*

אינדקסים *7*

class Bool *8*

If/else/elif *8*

Lists *9*

Loops *10*

Tuples *11*

נוסף *11*

Recitation 2 12

משתנים *12*

פונקציות - ערך ברירת מחדל *12*

Lecture 3: Refactoring, Set, dictionary, comprehension 13

Refactoring *13*

פונקציות מקוננות *14*

Hashing *14*

Sets *15*

Dictionaries *15*

Comprehensions *17*

נוסף *18*

Lecture 4: Aliasing, copying, more functions, slices, files, tuples, iterators 19

Iterators *19*

sort(), sorted(), reverse(), reversed() methods *20*
העתקת אובייקטים *20*
Recitation 4 22
זיכרון וקבצים בפייתון *22*
Mutable and Immutable *24*
Command Line Arguments *24*
Type Hints *25*
Lecture 5: Algorithms, runtime analysis 25
Analysis of algorithms *25*
סיבוכיות זמן ריצה *27*
Common Big O running time values table *29*
Recitation 5 30
Proof of correctness *31*
Lecture 6: Genexpressions, generators, memory considerations 33
חידוד *aliasing* – זיכרון ויעילות *33*
זמן ריצה ועלויות בקונטיינרים שונים – הרבה תמונות *34*
גנרטורים *38*
בעיות מיון *39*
נוסף *42*
Recitation 6 42
רקורסיה - מבוא *42*
חידודים *43*
Lecture 7: Recursion; Backtracking 43
רקורסיה *43*
Backtracking *44*
נוסף *46*
Lecture 8: Object Oriented Programming [OOP] 46
תכנות מונחה עצמים *46*
Encapsulation *50*
Inheritance *51*
נוסף *51*
Recitation 8 51
Lecture 9: More OOP; Linked data structures: Lists 54
Stack *54*
Linked Lists (Node) *55*
Recitation 9 60

Que 61
עצים 62
Lecture 10: Linked data structures: Trees, Graphs 64
Pre/In/Post Order 66
Lambda Functions - פונקציות אנונימיות 66
Map 67
Reduce 68
Filter 69
Recitation 10 69
Exceptions 69
נוסף 71
Lecture 11: Exceptions 72
Recitation 11 73
Iterable vs. Iterator 74
גנרטורים 74
נספח – צ'יטים לזמני ריצה 76

*נושאים חופפים מהרצאות שונות מאוגדים יחד (בערך)

Lecture 1: Introduction, functions, variables, types, expressions

Basic data types (values) in python

ערך מסוג מסוים נקרא *אובייקט* של ה- *class* שלו.

על מנת לבצע פעולות על נתונים נשתמש בפונקציות, אופרטורים (+, −,·,÷, ...) ושיטות (methods) שהן פונקציות שצמודות לערך מסוים.

`int`	מספרים שלמים
`float`	מספרים רציונליים
`string`	מחרוזות תווים
`bool`	אמת ושקר

האיברים של מחלקה מסוימת נקראים objects of the class. כל נתון או מבנה של נתונים הוא אובייקט של מחלקה כלשהי.

לכל אובייקט 3 מאפיינים:

- Class - למה הוא שייך
- Id - זהות, כתובת בזיכרון
- Value - הערך שלו

לאיזה טיפוס שייך ערך מסוים	`type('abc')`

כתובת בזיכרון	`id(3)`
אפשר לכתוב כדי לדעת מה פונקציה עושה. מחזיר גם את הדוקומנטציה שכתבנו עצמאית על פונקציות שהגדרנו.	`help(pow)`

משווה בין זהויות של אובייקטים ומחזיר ערך בוליאני	`'abc' is 'aBc'`
	או
	`'abc' is not 'aBc'`

פעולות חשבון בפייתון

כפל	*
העלאה בחזקה פונקציית החזקה	** pow(4.2,-6.7)
חילוק (נקבל בחזרה float)	/
חילוק שלם (מעגל למטה, התוצאה תמיד תהיה int)	//
מודולו – מחזיר את שארית החלוקה	%

Strings

חיבור מחרוזות ויצירת מחרוזת חדשה		+
יצירת מחרוזת חדשה ע"י שרשור של אותה מחרוזת מספר פעמים		*
מילות מפתח בפונקציית print:	איזו מחרוזת תודפס בין כל 2 ערכים (ברירת המחדל היא ' ' (רווח))	`sep = 'someStr'`
	איזו מחרוזת תודפס אחרונה (ברירת המחדל היא \n (ירידת שורה))	`end = 'anotheStr'`
	ירידת שורה	\n
	טאב	\t
	מדפיס \ (ביטול תפקידים מיוחדים)	\\
שלישיית גרשיים לירידת שורה באמצע		`'''hello` `world'''`
המשך כתיבה בשורה הבאה (גם לא בסטרינג)		\

String methods (*מהרצאה 2.1*)

`s.count(t)`	counts and returns the number of occurrences of string `t` in string `s`
`s.find(t)`	finds and returns the 1st position (starting from the left side), of string `t` in string `s`. If `t` is not in `s` it returns -1
`s.upper()`	returns a new string created from `s` with all letters as upper case, does not change the original string
`s.lower()`	returns a new string created from `s` with all letters as lower case, does not change the original string
`s.capitalaize()`	Return a copy of the string with its first character capitalized and the rest lowercased.

`s.title()`	Returns a title cased version of the string where words start with an uppercase character and the remaining characters are lowercase.
`str.replace(old, new [, count])`	Return a copy of the string with all occurrences of substring old replaced by new. If the optional argument count is given, only the first count occurrences are replaced.

`str.format(*args, **kwargs)`	`>>> name = "Eric"`
	`>>> age = 74`
	`>>> f"Hello, {name}. You are {age}."`
	`'Hello, Eric. You are 74.'`
	`>>> f"{2 * 37}"`
	`'74'`
	`>>> name = "Eric Idle"`
	`>>> f"{name.lower()} is funny."`
	`'eric idle is funny.'`

upper()	Converts a string into upper case
split()	Splits the string at the specified separator, and returns a list
replace()	Returns a string where a specified value is replaced with a specified value
count()	Returns the number of times a specified value occurs in a string
encode()	Returns an encoded version of the string
endswith()	Returns true if the string ends with the specified value
index()	Searches the string for a specified value and returns the position of where it was found
find()	Searches the string for a specified value and returns the position of where it was found
format()	Formats specified values in a string
startswith()	Returns true if the string starts with the specified value
strip()	Returns a trimmed version of the string

Members positions in sequences start from 0 : in 'abcde', 'a' is in position 0, 'b' in position 1, and 'e' in position 4.

Behaviour in the shell

הערות

- הערות שורה – קצרות, מתחילות ב- #
- הערה ארוכה יותר משורה אחת – עוטפים בשלושה גרשיים """ מכל צד.

הוראה חייבת להסתיים בשורה אחת. ניתן לשבור הוראה בכמה שורות אם משתמשים בסוגריים. בלי סוגריים אפשר להשתמש ב- /

Functions

יבוא פונקציה מספריה:

```
from math import sqrt, fractional, sin
```

הגדרה וקריאה לפונקציה:

The general syntax of a function:

```
def function_name(parameters_list):
    """docstring"""    # comment that explains the function
    statement(s)
```

If docstring is longer than a line, enclose it with triple quotes

```
# Example
def hello(name):
    '''prints a Hello greeting
    for any name sent to
    the function'''
    print('Hello', name)
```

```
# function call
hello('Ety')
```

Functions should be **short**.

Functions should have **1 distinct role**.

If a function has 2 roles, split it into 2 functions.

we must call the functions in the following form! ➔

```
if __name__ == '__main__':

    call_function()
```

(*מהרצאה* 2.5) גם אם לא כתבנו return הפונקציה עדיין מחזירה ערך – None.
אם כתבנו return, הפונקציה לא תגיע למה שכתוב אחריו כי הוא יוצא ממנה.

Return more values in a container

```
return [a, b, c] # a list
return a, b, c # a tuple
return a ** 2, b ** 3, a * b / (a - 1) # a tuple with the values of all these expressions
return {a, b, c} # a set ...
```

`dir(function_name)` : מראה את כל התוכן של הפונקציה או האובייקט (מחלקות):

```
['__annotations__',
 '__builtins__',
 '__call__',
 '__class__',
 '__closure__',
 '__code__',
 '__defaults__',
 '__delattr__',
 '__dict__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__get__',
 '__getattribute__',
 '__getstate__',
 '__globals__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__kwdefaults__',
 '__le__',
 '__lt__',
 '__module__',
 '__name__',
 '__ne__',
 '__new__',
 '__qualname__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__setattr__',
 '__sizeof__',
 '__str__',
 '__subclasshook__']
```

אם נרצה להראות למשל את ערכי ברירת המחדל של הפונקציה נשתמש ב:

```
function_name.__defaults__
```

נוסף

הצמדת הערך 8 למשתנה x	x = 8
מחיקת הערך המוצמד x (אובייקט שלא משתמשים בו ניתן למחיקה וכשיש בעיה בזיכרון יש גם תהליך בשם garbage collection שמוחק אותו)	del x
הצגת רשימת מילות המפתח ששמורות בפייתון	from keyword import kwlist
סופר כמה אובייקטים יש ברשימה	len(list_name)

Recitation 1

Python Turtle

בתחילת הקובץ	import turtle
בסוף הקובץ	turtle.done()
Put a ”pen” on the “canvas”	turtle.down()
Remove a “pen” from the “canvas”	turtle.up()
Move the turtle ***n*** steps forward	turtle.forward(n)
Turn turtle’s head in ***d*** degrees right	turtle.right(d)
Turn turtle’s head in ***d*** degrees left	turtle.left(d)

Lecture 2: Conditionals, list, strings, slicing, functions

ASCII and UNICODE

ASCII - טבלת תווים שמבוססת על האלפבית הלטיני.

UNICODE - הסטנדרט החדש, סימנים רבים, אותיות מ150 שפות.

Given `c`, a string representing one Unicode character, `ord(c)` returns an integer representing the Unicode place of character **`c`** in the Unicode table. (*`ord('a')`*)

the function `chr(i)` Returns the string representing the character whose Unicode position (in the table) is the integer `i`. It is the inverse of `ord()`.

אינדקסים

במחרוזת, רשימה, או כל מיכל,...

האינדקס של המקום הראשון הוא `0`. של האחרון הוא `len(s)-1`. המקום האחרון מיוצג גם עם `-1`.

Slicing:

- Slice of a string `s` is a substring (also of type `str`), from `s`
- Given `s` and two integers: `a,b` , `s[a:b]` provides a substring of `s` from positions `a` to `b-1`

<u>גישה לאובייקטים פנימיים - addressing inner objects within inner objects:</u>

```
L = [[1, 2], ['a', 'b']]
print(L[1])
print(L[0][1])
```

```
['a', 'b']
2
```

class Bool

הערכים היחידים במחלקה: `True, False`

אופרטורים: `and, or, not`

not	expression	truth value
	not T	F
	not F	T

or	expression	truth value
	T or T	T
	T or F	T
	F or T	T
	F or F	F

and	expression	truth value
	T and T	T
	T and F	F
	F and T	F
	F and F	F

Empty containers and zeros evaluate `False` and nonempty containers evaluate `True`.

Comparisons can be chained : `x < y <= z` is equivalent to `x < y and y <= z`.
(יעילות גבוהה יותר).

Testing membership in containers:

`in', `not In' check membership in containers, and return a boolean value: `True`, `False`.

If/else/elif

syntax:

```
if condition:
    indented block of statements
```

```
if condition:
    statement(s)

else:
    statement(s)
```

- The Boolean operations `or` and `and` always return the value of one of their operands:
- the one that that 'decides' the truth value of the whole expression:
- if the **`or`** expression is `True`: **`or`** returns the 1st `True` operand, else, it returns the last `False` operand
- if the **`and`** expression is `False`: **`and`** returns the 1st `False` operand,
  else, it returns the last True operand

```
if cond1:
  statement(s)

elif cond2:
  statement(s)

elif cond3:
  statement(s)
...

else:
  statement(s)
```

`or:`

`x or y` … if x is false, then y, else x.
`or` is a short-circuit operator: **it evaluates the 2nd arg only if the 1st one is `False`**.

`and`:

`x and y`… if x is false, then x, else y.
`and` is a short-circuit operator: **it evaluates the 2nd arg only if the 1st one is `True`**.

Conditional expressions:

```
if x >= y:
  max = x
else:
  max = y
```

=

```
max = x if x >= y else y
```

=

```
x if Cond else y
```

Lists

```
l = [`a', `b', 1, `cat']
```

```
l[0] = 'a'
```

```
l[1] + l[3] = bcat
```

```
In [1]: name=list('Einav')
        name

Out[1]: ['E', 'i', 'n', 'a', 'v']
```

Step in slice: slice land

`l[start:end:step]`

```
print(L9[1:6:2]

[1, 3, 5]
```

רשימה מהסוף להתחלה: `l[::-1]`

`del L[0]`	מסיר איבר מהרשימה באמצעות אינדקס (בדוגמה: באינדקס 0)
`L.append('blue')`	מוסיף איבר חדש לסוף הרשימה
`L.remove('red')`	מסיר איבר מהרשימה באמצעות הערך שלו
`L.extend(['x', 'y'])`	הארגומנט חייב להיות קופסה. מפרק את הקופסה, לוקח את האיברים ושם אותם אחד אחד בסוף הרשימה
`L.insert(1, 'Lior')`	באיזה מיקום להוסיף ומה להוסיף - (where, what)
`min() , max()`	
`sum`	
`list.pop(obj=list[-1])`	Removes and returns last object or obj from list
`list.reverse()`	Reverses objects of list in place
`list.sort([func])`	Sorts objects of list, use func to compare, if given

```
a, b, *c = range(5)
print(a,b,c)

0 1 [2, 3, 4]
```

```
lst = [1, 2, 3, 4]
lst[1:3] = ['a', 'b', 'c']
lst

[1, 'a', 'b', 'c', 4]
```

list is assigned to 2nd and 3rd places in lst, instead of ints 2, 3.

אם נשים ב- [0:0] ידחוף בתחילת הרשימה בלי לדרוס.

בנ"ל אם נשים פשוט בשני צידי הנקודותיים אותו מספר.

lst[:] = … מחליף הכל או מנקה את הרשימה.

```
# using Tuple Unpacking, indexing and not slicing
lst = [1, 2, 3, 4, 5, 6]
lst[5], lst[2], lst[0] = {'iosi', 'ety',"apple"}
lst # set ה"א סדר" בגלל ה
```

```
['ety', 2, 'iosi', 4, 5, 'apple']
```

```
# inserting the right number of elements from a container
L3 = [1, 2, 3, 4, 5, 6, 7, 8, 9]
L3[1 : 8 : 2] = ['A', 'B', 'C', 'D'] # objects to be inserted come from a list
L3
```

```
[1, 'A', 3, 'B', 5, 'C', 7, 'D', 9]
```

Loops

`while` condition: indented block of statements	`For` aVar `in` iterable: statement(s)

- iterable - a collection of elements (any iterable container)
- aVar - the loop variable
- the statements block is executed, `len(iterable)` times.

Finding the GCD of two numbers:

The greatest common divisor (GCD) of two integers a , b is the largest number that divides both without remainder. Euclid's algorithm:

`if r = a % b, then gcd(a, b) = gcd(b, r).`

`It ends when gcd(a, 0) = a.`

When a = 0, it takes one iteration to return b The result will have the sign of b (so that when b is divided by it, the result comes out positive). (unless b == 0)

גם אחרי לולאות אפשר לשים `else`. מבוצע אם הלולאה הסתיימה באופן טבעי (ולא עם `break` למשל) או אם הלולאה בכלל לא התחילה.

Break:
מוציא את התוכנית מחוץ ללולאה.

Continue:
דילוג על החלק של הלולאה שמגיע אחרי, התחלה חדשה של האיטרציה.

לולאה אינסופית: break ידני – ctrl+c.

Range:

`range(stop)`: all integers from 0 to (stop-1)
`range(start, stop)`: all integers from start to (stop-1)
`range(start, stop, step)`: all integers from start to stop-1 by step step

Range is an **immutable** sequence type, and it behaves as an **iterable container.**

<u>לולאות מקוננות</u>

```
For aVar1 in iterable:
    For aVar2 in iterable:
                statement(s)

    statement(s)
```

על כל ריצה של ה- for החיצוני, ה- for הפנימי רץ מהתחלה ועד הסוף.

Tuples

Container – סוג של רשימה בלואה. דומה לרשימה מבחינת הורסטיליות. אפשר לשים מה שרוצים ובאיזה סדר שרוצים. אבל בניגוד לרשימה, ברגע שה- tupple נוצר אי אפשר לשנות אותו. כמו סטרינג. אם בתוך ה-tupple יש רשימה אפשר לשנות אותה מבפנים. אבל כאיבר אי אפשר לגעת בה.

```
tup = ( 12, "bcd", 3.45, (1, 2), "efg" ) # with parentheses
tup
```

```
tup1 = -1, 2.9, "iosi", [22, 3] # without parentheses
tup1
```

בשקובעים את ה-tupple זה נקרא tupple packing.

– Sequence unpacking

Sequence unpacking

`x, y, z = t`
This is called `sequence unpacking` and it works for `any sequence` on the right-hand side.

```
x, y, z = t
print(x, y, z)
```

```
45 32 ety
```

Sequence unpacking requires that there are as many variables on the left side of the `equals sign` as there are elements in the sequence.

Note that `multiple assignment` is really just a combination of `tuple packing` and `sequence unpacking`.

```
a, b, c, d = 11, 'a', [ -3], ( 2,)
print(a, b, c, d)
```

```
11 a [-3] (2,)
```

Sequence, Mutable: list
Sequence, Immutable: string, tuple, range

נוסף

קבלת קלט מהמשתמש. אוסף אותו כמחרוזת. המשתמש מכניס ולוחץ אנטר.	`n = input([str prompt])` `n`
המרה לסוג אחר	`int(input('pls enter an int: '))` `float(input('pls enter a float: '))`

מקבל קלט והופך אותו לביטוי בפייתון (למשל אפשר להכניס 5+3 ואז הוא יחזיר 8	`eval(expression)`
	`n = eval(input('a float please:'))`
	`n`

עובר הלאה, לא מבצע כלום, שומר מקום עד שנוסיף משהו	`def funct():` `    pass`
נותן ערך לביטוי	:=

האם a הוא מסוג x? מחזיר ערך בוליאני (*דוגמאות בסוף הרצאה 2.5*)	`isinstance(a,x)`

Recitation 2

משתנים

כתובת בזיכרון שמשויך אליה איזשהו שם עם ערך בפנים.

- set, השמה – קביעת הערך במשתנה
- שימוש, get - קבלת הערך מהמשתנה
- מוסכמות:
  - אותיות קטנות
  - אם יש כמה מילים, שימוש בקו_תחתון
  - שמות עם משמעות
  - עבור משתנים קבועים (שכנראה לא נשנה במהלך התוכנית) – אותיות גדולות (*PI, ERROR_MASSAGE, ...*)

Scopes:

טווח הכרה של התוכנה. משתנים גלובליים ומשתנים מקומיים.

```
X = 2       #global
Def function():
          x=2       #inner
```

שינוי ערך של משתנה גלובלי בתוך פונקציה – לא מומלץ, לא להשתמש בקורס הזה אלא אם כן נאמר אחרת. אבל עושים זאת באמצעות `global`:

```
numcalls = 0
def square(x):
    global numcalls  # to get permission to change numcalls value inside square()
    numcalls = numcalls + 1
    return x * x
```

כשרוצים להגדיר בתוך פונקציה משתנה שהתוכנה תכיר גם בחוץ – `global num_student` – מומלץ להמנע.

אין צורך להשתמש ב- global כשמכניסים רשימות לתוך פונקציה כי הן מועברות כ- pointer. אז שינוי בתוך הפונקציה משנה אותן. כי הן mutable. אבל:

```
L2 = [7, 8, 9]

def f7():
    L2 = [] # local
    L2.append(10)
```

כשמגדירים עם '=' זה הופך למשתנה מקומי ולא נוגע בחיצוני.

פונקציות - ערך ברירת מחדל

```
def shoresh(number, root=2):
    print(number ** (1/root))
```

ערכי ברירת המחדל צריכים להיות מוגדרים בצד הימני ביותר.

אפשר גם להתייחס אליו כאילו הוא לא קיים:

```
shoresh(64)  # default value was used
>> 8
```

```
shoresh(64, 3) # user value was used, not the default
>> 4
```

return מוציא סקופ החוצה מהפונקציה והופך אותו לגלובלי.

Important warning: The default value is evaluated only once. This makes a difference when the default is a mutable object such as a list, dictionary.

שינוי ערכי ברירת מחדל:

```
f9.__defaults__
```

```
(3, [3, 4, 5], {'a', 'cc', 'r'}, {11: 22, 33: 55, 'Iosi': 'Lo ety'})
```

```
f9.__defaults__[1][0] = 'Malka'
f9.__defaults__[3][33] = 'Python'
f9.__defaults__[3]['apple'] = 'grapes'
```

```
f9.__defaults__
```

```
(3,
 ['Malka', 4, 5],
 {'a', 'cc', 'r'},
 {11: 22, 33: 'Python', 'Iosi': 'Lo ety', 'apple': 'grapes'})
```

שמירה על ערך ברירת מחדל: לא לשים ערך שהוא mutable. לשים None או כל דבר שהוא לא mutable.

```
def foox(x, items = None,a=7,b=9):
    if items is None:
        items = []
        items.append(x)
    return items
```

We can use an indefinite number of parameters by using a final parameter name of the form *args:

```
def myFun(a, b, *numbers):
    S = 0.0
    print(type(numbers))

    if len(numbers) > 0:
        S = sum(numbers)

    return a, b, numbers, S
```

```
myFun(1, 0, 3, 4, 5, 6, 7) # 3, 4, 5 enter as  a tuple in numbers
```

```
<class 'tuple'>

(1, 0, (3, 4, 5, 6, 7), 25)
```

אם שמים את הכוכבית אפשר להכניס כמה ארגומנטים שנרצה ועוטף אותם בטאפל.

Refactoring

Keep functions shorter than a page, (easier to grasp), it's hard to keep track of indentation on more pages.

The median size of our functions is 15 lines; the mean is 19 lines.

חוק דה מורגן:

למערכת קצת יותר קשה לחשב not והוא גם יותר מסורבל. פחות יעיל.

```
print (not(a < b and c >= d) == (a >= b or c < d))
```

Refactoring tips

Instead of writing :

```
if n > 0:
    return True
else:
    return False
```

we could write:

```
return n > 0
```

Syntax note

`if st` with skinny blocks (1 st) like:

```
if cond :
    statement1
else:
    statement2
```

can also be written as :

```
if cond : statement1
else: statement2
```

```
if cond : statement1
else: statement2
```

try and except:

אם שמנו משהו שמפיל את התוכנית היא לא נופלת אלא הולכת לקטע קוד שכתוב מתחת ל- except.

```
# index() raises an exception if target is not found therefore we use a try/except construct
# try/except is presented in the other part of the lecture

def indx3(lst, target):
    try:
        return lst.index(target)
    except:
        return -1
```

פונקציות מקוננות

```
# Example 1
def f(x,y):
    """Calculate Percentages from the total"""
    def g(k):
        return (k * 100.0) / (x + y)
    print('The results are:', g(x), g(y))
```

קריאה ל- f, מפעילה את g שמוגדרת בתוכה, ראשית על x ואז על y.

Hashing

פונקציה hash היא פונקציה שניתן לתת לה טקסט באיזה אורך שרוצים והיא מחזירה מספר שלם בגודל קבוע (20 ספרות או 19 ספרות במחשב 64bit ו9 או 10 ספרות במחשב 32bit).
אם נעשה hash לשני טקסטים וזה אותו מספר זה אומר שהם זהים. לדעת אם שני סטרינגים הם אותו דבר בלי לדעת את הערכים שלהם.

בפייתון יש אובייקטים שיש להם hash number ובאלה שלא.

אז מי לא hashable?
אובייקטים שהם mutable, כלומר, שאפשר לשנות אותם, למשל, לרשימה אין hash number כי אפשר לשנות אותה. אז lists, sets, dictionaries,... ל- tupple שמכיל בתוכו רשימה אין hash number, הוא לא "טהור".

מי hashable?
string, tupple, Boolean values, numbers הם קבועים ולכן להם יש hash number.

בדיקה האם אובייקט הוא hashable: `isinstance("abc", Hashable)`

לשים לב שטאפלים שבתוכן איברים mutable כמו רשימות למשל, מחזירים שהם hashable למרות שהם לא.

Sets

sets (mutable) / frozensets (immutable) are unordered collections of unique objects.
Sets/frozensets are unordered, and cannot be indexed by numbers.

בפייתון set הוא אוסף לא מסודר של אלמנטים ייחודיים, מה שאומר שאף אלמנט לא יכול להופיע יותר מפעם אחת בקבוצה ואי אפשר לגשת לפריטים באמצעות מספר האינדקס שלהם.

אם נהפוך רשימה שמכילה אלמנטים שמופעים יותר מפעם אחת לסט יכתבו רק אלמנטים ייחודיים. סדר האלמנטים לא בהכרח ישמר.

נכתבים עם סוגריים מסולסלים {} או עם `set()`.

`.add`, `.remove`	סטים הן mutable – ניתן להוסיף ולהסיר איברים
`.discard`	מסיר אלמנט מהרשימה, לא נותן שגיאה אם האלמנט לא נמצא
`.pop()`	מסיר אלמנט אקראי מסט נתונים ומחזיר אותו
`.clear()`	מסיר את כל האלמנטים מהסט
`x.update(y)`	אם x,y סטים, `x.update(y)` ממזג אותם
`s1-s2`	הפרש - the result is a set whose elements are in s1 but not in s2
`s1|s2`	איחוד
`s1&s2`	חיתוך
`s1^s2`	או בזה או בזה אבל לא בשניהם

Sets do not support indexing, slicing, or other sequence-like behavior

Frozen set : immutable, hashable.

Frozen set יכול לשמש כאיבר בסט אחר או כמפתח במילון.

```
x = frozenset('abcd') # arg is a string
print('x = ', x)

y = frozenset([1, 2, 3, 4]) # arg is a list
print('y = ', y)

z = frozenset(set('Iosi')) # arg is a set
print('x = ', z)
```

Dictionaries

Mutable

A dictionary is an unordered associative container that maps keys to values

key	value
'a' :	[1,2,3]
12 :	'abc'
(2,'b'):	{1,2,3}
'abc' :	{1:11,1:22}

מילון זאת קופסה עם זוגות.

איבר במילון: `key : value`

ה- values יכולים להיות אותו דבר אבל כל מפתח צריך להיות שונה.

המפתחות במילון הם יחידים. אין את אותו מפתח פעמיים. בנוסף, הם hashable. סטים יכולים להיות מפתחות.

יצירה של מילון:

```
# Create an empty dictionary!
A = {}
print('A =', A)

# Set initial key:value pairs
B = {'a' : 'alef', 'd' : 'dalet', 'g' : 'ghimel'}
print('B =', B)

C = {(1,2) : False, "bar" : [1, 7, 3], 2.99 : "almost three"}
print('C =', C)
```

`# Get the value associated with Key` `C[2.99] # "almost three"`	Dictionary values are accessed by specifying their key.
`dict_name['iosi'] = 21`	Insert a new item. יוסי הוא המפתח, 21 הוא הערך שצמוד אליו. באותו אופן נוכל לשנות את הערך שצמוד למפתח.
`update()` `A.update(B)`	מיזוג של שני מילונים – עדכון מילון עם דאטה ממילון אחר. בדוגמה – A מילון, B מילון
`del dict_name[key]`	מחיקת איבר מהמילון
`dict_name.clear()`	מסיר את כל האיברים מהמילון
`dict_name.items()`	Returns a view of the items of the dictionary (`key, value`) pairs
`dict_name.keys()`	Returns a view of the keys of the dictionary
`dict_name.values()`	Returns a view of the values of the dictionary

`get(Key, NotFoundValue)`
`d = {1:11, 2:22}`
`d.get(1)`
 	
`d.get(3,'ain"t in yet')`
	אם קוראים לפתח והוא לא נמצא במקום הודעצת שגיאה אפשר להחזיר ערך שנקבע מראש. למשל הודעה "הערך לא נמצא".
	if Key is not found in the dictionary: `get(Key, NotFoundValue)` returns the NotFoundValue (mostly a string), instead of raising an exception (error). NotFoundValue has a default value of None.

`pop( key [, default ] )`
`x = d.pop('iosi', "sorry")`
`print(x)`
`print(d)`
 	
`sorry`
`{1: 11, 2: 22}`
	להסיר מפתח מהרשימה וגם לקבל החזרה שלו. גם כאן, אם הערך לא נמצא אפשר להחזיר משהו. למשל "סליחה".

`popitem()`	לקבל גם את המפתח וגם את הערך. Remove and return a (key, value) pair from the dictionary. Pairs are returned in LIFO order (Last In First Out). `popitem()` is useful to destructively iterate over a dictionary. If the dictionary is empty, calling popitem() raises a KeyError.
`function(**dict_name)`	הכוכביות מפרקות את המילון. שולחות את האיברים בנפרד ולא בתוך אריזת המילון.

<u>יצירת מילון עם הפונקציה `dict(..)`</u>

Function `dict(..)` needs to 'understand' which are the pairs `key : val`, so its arguments have to 'reflect' a group of pairs:

```python
a = dict() # creates an empty dict
b = {'one': 1, 'two': 2, 'three': 3} # straightforward
c = dict(one = 1, two = 2, three = 3) # with assignments..

d = dict([('two', 2), ['one', 1], ('three', 3)]) # with tuples
e = dict({'three': 3, 'one': 1, 'two': 2}) # with another dict
```

אפשר לשים בפונקציה רשימה של רשימות שבכל אחת זוג איברים (מפתח וערך), כנ"ל רשימה של טאפלים, רשימה של סטים, רשימה של containers מסוגים שונים.

Comprehensions

List Comprehenshions

```python
l = [exp(i) for i in seq if cond(i)]
```

```python
squares = []
for x in range(10):
    squares.append(x ** 2)

print('A list of the squares of 0-9 :', squares)
```

⟺

```python
squares1 = [x ** 2 for x in range(10)]
print('A list of the squares of 0-9 :', squares1)
```

A list comprehension consists of:

- brackets containing an expression followed by a for clause,
- then zero or more for, or if clauses.

The result will be a new list resulting from evaluating the expression in the context of the for and if clauses which follow it.

לולאות מקוננות:

```python
lst = []
for x in [1, 2, 3]:
    for y in [3, 1, 4]:
        if x != y:
            lst.append((x, y))

print('lst :', lst)
```

⟺

```python
Lc = [{x, y} for x in [1, 2, 3] for y in [3, 1, 4] if x != y]
print('Lc :', Lc)
```

מעגל ל- i ספרות אחרי הנקודה:

```python
from math import pi
Lc8 = [str(round(pi, i)) for i in range(1, 11)] # ['3.1', '3.14', '3.142', '3.1416', '3.14159']
print('Lc8 :', Lc8)
```

set Comprehenshions

The elements of such sets must be Hashable.

```
ST = {x*2 for x in 'abracadabra'}
print('ST =', ST)
```

```
ST1 = {(x + y, y - 1) for x in range(3) for y in range(2) if x != y}
print('ST1 =', ST1) # {(3, 0), (1, 0), (2, -1), (1, -1)}
```

```
ST2 = {x + y for x in 'iosi' for y in 'moshe' }
print('ST2 =', ST2)
```

dictionary Comprehenshions

The keys of such dictionaries must be Hashable.

- Use enclosing braces,
- Separate the pair by a colon,
- The "raw materials" come in a container, packed as 2 elements inner containers.
- Must: 1st element of the 2 seater is hashable (to become a key).
- The result is a dictionary.

```
D1 = {i:j for i, j in [(1, 2), ('a', 'b'), (3, 'c'),(1,11)]}
print('D1 :', D1)
```

```
D1 : {1: 11, 'a': 'b', 3: 'c'}
```

נוסף

השוואה בין רשימות או בין טאפלים

and remember that `(x, y, z) == (1, 1, 1)` checks values equality
and `(x, y, z) is (1, 1, 1)` checks addresses equality.

Binet formula for the n-th Fibonacci number:

$$f_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

הרצאה 3.1 הדפסת משולש כוכביות, מספרים - "ללמוד את הלולאות טוב..." (-אריה)

```
# using int() to cast float result
n = 10
for i in range(1, n):
    print (int(i * ((10 ** i) -1) / 9))
```

```
1
22
333
4444
55555
666666
7777777
88888888
999999999
```

```
# using str()
for i in range(10):
    print(str(i) * i)
```

```
1
22
333
4444
55555
666666
7777777
88888888
999999999
```

```
# not nested and not numbers
n = 4
for i in range(n):
    print('*' * (i + 1))
```

```
*
**
***
****
```

דוגמה לכפל מטריצות:

```python
# inner list in a,b,c represent columns
a = [[1, 0, 0], [0, 1, 0], [0, 0, 1] ]  # identity matrix 3x3
b = [[1, 2, 3], [4, 5, 6], [7, 8, 9] ]
c = []

for irow in range (3):
    c.append ([])          # append inner empty lst
    for icol in range (3):
        c[irow].append (0) # c is initialized with zeros
        for k in range (3):
            c[irow][icol] += a[irow][k] * b[icol][k] # the multiplications & additions are done here
print (c)
```

```
[[1, 4, 7], [2, 5, 8], [3, 6, 9]]
```

(בתרגול 3 לא היה שום דבר מעניין)

`*import random`

*Ceasar Code

Lecture 4: Aliasing, copying, more functions, slices, files, tuples, iterators

Iterators

איטרטורים משמשים לחסכון בזיכרון.

איטרטור הוא אובייקט שמייצג זרם של נתונים שמחובר לקופסה. באמצעות פונקציה `next()` אפשר כל פעם לקבל את האיבר הבא.

הפונקציה `iter()` מייצרת את הצינורית הזאת. מחזיר איטרטור.

```python
L = [1, 2, 3]
it = iter(L)
```

```python
next(it)
```

```
1
```

```python
s = 'iosi'
itush = iter(s)
```

```python
print(next(itush))
```

```
i
```

```python
# iterator to a set
zet = {(), 'ab-)&8', -1.23, (3, )}
it2zet = iter(zet)
```

```python
next(it2zet)
```

```
'ab-)&8'
```

```python
dikt = {"hi":45, ("pas3$",):[11,-2]}
it2dikt = iter(dikt)
```

```python
next(it2dikt)
```

```
'hi'
```

ניתן להכניס ל- `iter()` גם קריאה לפונקציה.

איטרטורים:

`enumerate()`
`seasons = ['Spring', 'Summer', 'Fall', 'Winter']`
`en2Lst = enumerate(seasons)`
`next(en2Lst)`
`(0, 'Spring')`
	יוצר טאפלים של זוגות - האינדקס והערך של איברים מהקונטיינר שהכנסנו אליו.

`zip()`
`z = zip('abcd', 'xyzw', [1, 2, 3, 4])`
`next(z)`
`('a', 'x', 1)`
	מחזיר טאפלים כך שהטאפל ה- i מכיל את האיבר ה- i מכל איבר בקונטיינר.

`zip(*k)`	(K הוא רשימה). *באופן כללי בזימון של פונקציה כותבים כוכבית ביחד עם איזשהו קונטיינר והוא מתפרק ("לשלוח את האיברים בלי השקית").* באן מה שקורה זה שנקבל בחזרה את המקור שהיה לפני הזיפ המקורי.

```
print(k)
```
```
[('a', 1), ('b', 2), ('c', 3)]
```
```
print(list(zip(*k)))
```
```
[('a', 'b', 'c'), (1, 2, 3)]
```

```
z = zip(['a', 'b', 'c'], [1, 2, 3])
```

sort(), sorted(), reverse(), reversed() methods

`sort()`	(עבור רשימות בלבד) ממיינת רשימה של מספרים. הרשימה המקורית נעלמת. לא מחזירה כלום – None.
`reverse()`	(עבור רשימות בלבד) לוקחת רשימה ומחליפה בין הפוזיציות. מקום ראשון הופך לאחרון,.... מחליפים בזוגות כלפי האמצע. גם כאן הרשימה המקורית נעלמת. לא מחזירה כלום – None.
`sorted()`	לא רק של רשימות. מחזירה רשימה ממוינת ושומרת על המקור. אפשר לעשות sorted גם ל- tupple, string, set,... תמיד נקבל בחזרה רשימה. לוקחת איברים של ה- tupple (למשל), שמה ברשימה וממיינת. יכול להיות עם כל ה- iterable containers.
`reversed()`	לא עובד על סטים ועל מילונים כי שם אין שום סדר אז אין משמעות להחליף את המיקומים של האיברים. כמו sorted, שומר על המקור.

- Python uses an algorithm called **Timsort** which is like **merge-sort** that you will learn next week.
- This algorithm works in **O(nlogn)**
- The function **sorted()** returns a **new** sorted list

```
>>> sorted([5, 2, 3, 1, 4])
[1, 2, 3, 4, 5]
```

- **list.sort()** method of List, sorts the list **in-place** (and returns None to avoid confusion).

```
>>> lst = [5, 2, 3, 1, 4]
>>> lst.sort()
>>> lst
[1, 2, 3, 4, 5]
```

העתקת אובייקטים

לכל אובייקט יש כתובת ייחודית בזיכרון: `id(object)`

Aliasing – לתת יותר משם אחד לאותו אובייקט.

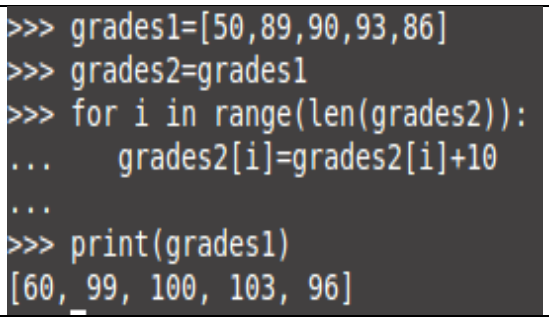
למשל:

```
a = 100
b = a
```

Using the assignment operator (=) always copies just the identity of the variable on the right to the variable on the left (no matter what value the right variable has) – so the value itself is not being copied.

When we modify the variable's value, we making a new identity for that variable.

הוספה של ערך נוסף ב- immutable כן משנה את המיקום בזיכרון.

		grades 1 ו- grades 2 מצביעים לאותו מקום. מכיוון ששתיהן רשימות אם נשנה אחת נשנה גם את השנייה, הן למעשה אותו דבר.

בשמכניסים רשימות לתוך פונקציה הן מועברות ב- pointer. כל שינוי שנעשה בפנים ישפיע על הרשימה שבחוץ (mutable). כדי להימנע מזה נשתמש ב- copy.

Shallow and Deep copying

Shallow copy - Creates a new container and fills it with references to the objects inside the original – Immutable items are copied by value.

- Simple data types – by value
- Compound data types - by reference!

Shallow copy - לוקח את הדברים הפשוטים ושם במקום אחר בזיכרון. את הדברים הלא פשוטים – שמכילים בתוכן דברים אחרים, עושה רק פוינטר למקום בזיכרון ואז אנחנו עלולים לשנות גם את המקור.

```
from copy import copy
c = [[300, 400], {"Huji":(21, -8), "Tau":range(48)}]
e = copy(c)
```

`stu1=stu2[:]`

Deep copy - In deep copy the data is copied over. Create a new version of the same values but with new references. The result is different from the result of shallow copy. Advantage: A and B do not depend on each other. Disadvantage: A slower and more expensive copy.

Deep copy – מעתיק הכל למקום חדש.

```
from copy import deepcopy
c = [[1], {2, 4}]
f = deepcopy(c)
```

`from copy import deepcopy`

`stu2=deepcopy(stu1)`

In Lists == works similarly to deepcopy and compares the values of two lists

- In mutable items we distinguish between three cases:
- Assignment (=):
  - Assign a second name to the same identity.
- Shallow copy ([:]):
- Copy all the fields (values for simple types and references for memory addresses).
- Deep copy (copy.deepcopy):
- Copy (recursively) all the values from one container to the other.

הערות

- *About some operators that combine containers - להסתכל פשוט בקובץ של הרצאה 4.2*
- *החלק על זיכרון וקבצים כתוב בסיכום תרגול*
- *Counter בהרצאה 4.5 לא בקורס הפעם*
- דילגנו על 4.4

Recitation 4

זיכרון וקבצים בפייתון

יש מגבלה על כמה מידע container יכול להכיל.

על מנת לשמור מידע נשתמש בקבצים. Sequence מסודר של ביטים. כל המידע נשמר שם בניגוד למשתנה שיכול להידרס ואפשר לאכלס בקבצים יותר מידע והרבה סוגי מידע.

קבצים: API:

`open(filename,[mode])` - פונקציה שמורה בפייתון שמחזירה קובץ. ה- filename הוא מחרוזת שמייצגת את מיקום הקובץ במערכת הקבצים.

כשנפתח קובץ נגיד איך אנחנו רוצים להשתמש בו לטובת הרשאות והמנעות מטעויות:

- 'r' – read. הערך ברירת מחדל ב- mode - קובץ לקריאה
- 'w' - write. אופציה זו לא מוסיפה לקובץ אלא דורסת אותו וכותבת מעל
- 'a' – add. מוסיפה דברים לקובץ, לא דורסת
- 'r+' – גם קריאה וגם כתיבה

<u>מתודות כתיבה</u>:

`f.write(s)`	כותבת לתוך הקובץ f ומחזירה את מספר התווים שכתבתי. בשביל לא לקבל שגיאה צריך לפתוח לפני זה את הקובץ ב- Write או add. כשכותבים ב- write mode אין ירידת שורה באופן אוטומטי כמו ב- print.
`f.write('\n'.join(list_of_strings))`	אם רוצים לרדת שורה

```
f = open("file.txt", 'a')
L = ['Happy','new','year']
n = f.write('\n'.join(L))
```

```
Happy
new
year
```

`Writeline` מצפה לקבל רצף של מחרוזות.

```
f = open("file.txt", 'a')
     my_strings = ['Don', 'key']
f.writelines(my_strings)
```

```
Donkey
```

`f.close()` – אחרי שנגמר השימוש בקובץ סוגר את הגישה אליו ומשחרר משאבים. אם שוכחים לעשות את זה, כדי לוודא שהתוכנה לא תקרוס ולוודא שאנחנו נגשים לקבצים בצורה בטוחה:

```
with open(file_path, 'w') as our_open_file:
   # we can work here with the open
   # file and be sure it is properly
   # closed in every scenario
```

נסגר באופן אוטומטי לאחר היציאה מההזחה.

על מנת לבדוק אם הקובץ *f12* סגור:

```
f12 = open('a.txt', 'w')
f12.write('hi')
f12.close()

if f12.closed :
    print("It's closed, I'm late")
```

אם ננסה לעבוד על קובץ סגור: `ValueError: I/O operation on closed file`

<u>מתודות קריאה:</u>

`f.read(n)`	אם מכניסים מספר n קוראת את מספר התווים הזה. אם לא, תקרא את כל הקובץ. מחזירה מחרוזת.
`f.readline()`	קורא שורה אחת
`f.readlines()`	Read all lines in the file - returns a list of strings (list of the file's lines). כל פעם שנקרא לה תקרא את השורה הבאה.
`f.tell()`	מחזירה את המיקום הנוכחי בתוך הקובץ.
`list(f)`	מכניס את כל השורות לרשימה

<u>ייבוא קבצים:</u>

`import foo` `foo.func1()`	
`import foo as my_foo` `my_foo.func1()`	
`from foo import func1,func2` `func1()`	
`from foo import *` `func1()`	מייבא את הכל מהקובץ. עלול ליצור טעויות ודריסת משתנים.

`if __name__ == "__main__":`	מריץ רק אם אני בקובץ הראשי.

```
f.read([n])         # Reads at most n characters (from a text file).
f.readline([n])     # Reads a single line of input up to n characters.
                    # If n is omitted, this method reads the entire line.
f.readlines([size])# Reads all the lines and returns a list whose members are the lines.
                    # size optionally specifies the approximate number of characters
                    # to read on the file before stopping.

f.write(s)          # Writes string s.

f.writelines(lines)# Writes all strings in sequence lines.

f.close()           # Closes the file.
```

Mutable and Immutable

Objects whose value can be changed are said to be mutable

An object's mutability is determined by its type:

- Immutable - numbers, strings, ranges and tuples
- Mutable - lists, sets, dictioneries

מחרוזת לא ניתנת לשינוי ברגע שעשיתי לה השמה. אפשר לעשות השמה חדשה למקום חדש בזיכרון.

ברשימה למשל אם נשנה את אחד הערכים המיקום של הרשימה בזיכרון ישמר. רשימה היא mutable.

Immutability – גישה בטוחה, ה- interpreter מוותר על כל מיני חישובים.

Object	Mutable or Immutable?
int, float (a = 1, a = 1.5)	Immutable
string (a = "hello")	Immutable
range (a = range(10))	Immutable
tuple (a = (1,2))	Immutable
list (a = [1,2])	Mutable

Command Line Arguments

לפעמים אנחנו רוצים להריץ את הקוד שלנו מבחוץ. דרך לשנות את הארגומנטים כל פעם, דרך הטרמינל.

כדי להריץ את התוכנית דרך הטרמינל נשתמש ב:

```
python3 prog.py arg1 arg2 arg3 ...
```

איך בתוך הקוד נשתמש בארגומנטים האלה?

נייבא את החבילה sys ונשתמש בפונקציה argv למשל באופן הבא:

```
import sys

args = sys.argv

program_name = args[0]

argument1 = args[1]

argument2 = args[2]
```

לשים לב שהארגומנטים הם סטרינגים!

Type Hints

לפעמים כדאי להגיד לפייתון לפני מה אני רוצה שהמשתנה יהיה.

- הקוד יהיה יותר נוח לקריאה
- עוזר לדוקומנטציה
- מכריח לחשוב על איזה טייפים כל פונקציה צריכה לקבל

בפייתון אין בדיקה של סוגי טייפים. גם אם אמרנו ש- x צריך להיות int, אם נשים שם משהו אחר הוא ייתן לנו. יש חבילות שאפשר להתקין שזה לא יהיה ככה.

```
varname:type = value
```

```
def func_name(var:type) → return_type
```

```
def calc_circle_area(radius: float) -> float:
    return 3.14 * radius * radius
```

```
>>> num1: int = 2
>>> print(num1)
2
>>> num1: int = 2.5
>>> type(num1)
<class 'float'>
```

Typing Module – ספריית טייפינג:

```
from typing import List

# Grades_List is the type of a list of floats.
Grades_List = List[float]

def class_average(grades: Grades_List) -> float:
    return sum(grades)/len(grades)
```

```
>>> type(Grades_List)
<class 'typing._GenericAlias'>
```

```
from typing import Any

def print_whatever(something: Any) -> None:
    print(something)
```

Lecture 5: Algorithms, runtime analysis

Analysis of algorithms

קטע קוד הוא יעיל אם הוא מהיר וגם לא תופס הרבה מקום בזיכרון.

הדגש העיקרי שלנו הוא מהירות הביצוע.

For any algorithm we create/study, we want to know:

- If it produces the correct answer (`Correctness`)
- If it is efficient (`Efficiency` in using resources)

דוגמה - חיפוש אובייקט ברצף באמצעות לולאה. המקרים האפשריים ליעילות:

- הכי טוב – האובייקט נמצא בהתחלה ומצאנו אותו מהר
- הכי גרוע – האובייקט נמצא בסוף אז מצאנו אותו בין האחרונים
- אמצע – האובייקט איפשהו באמצע

דוגמה נוספת – מי הגדול ביותר ומי הקטן ביותר או מי לא הכי קטן ולא הכי גדול. כאן נצטרך לבדוק את כל האיברים – אין הכי טוב והכי גרוע.

יעילות הביצוע של פונקציה לרוב תלויה בגודל הקלט. אבל לא תמיד. למשל:

```
def f4(n):
    return 'Hi'
```

אין שום קשר לדאטה שקיבלנו.

<u>חיפוש לינארי</u>

עד שהאובייקט נמצא ואז תצא החוצה. יש בדיקה של if בכל סיבוב.

```
def linear_search(seq, obj):
    ind = 0
    while ind < len(seq):
        if obj == seq[ind]:
            return ind
        else:
            ind += 1

    return None
```

<u>sentinel linear search – חיפוש לינארי זקיף</u>

שולף את האיבר האחרון בסדרה ושומר במשתנה.

שותל במקום האחרון בסדרה את האובייקט שאני מחפש. (יכול להיעשות רק ברשימה. במחרוזת או טאפל צריך להרכיב סדרות חדשות). האובייקט בסוף הסדרה משמש כזקיף כדי שהלולאה תעצור.

בלולאת while אנחנו גם משווים לאובייקט וגם מקדמים את i באיטרציה. ברור לנו שהלולאה הזו תעצור מתישהו כי שמנו את האובייקט האחרון להיות האובייקט שאנחנו מחפשים.

אחרי שהלולאה מפסיקה מסיבה זו או אחרת מחזירים את האיבר האחרון למקומו בסדרה.

כעת מגיעים לif היחיד בתוכנית. אם האובייקט נמצא בסדרה תחזיר את מקומו ואחרת none.

לולאה דקה יותר ו- if עם שתי בדיקות. יותר יעיל מהחיפוש הלינארי.

```python
def sentinel_linear_search(seq, obj):
    n = len(seq) - 1 # n is the index of the last element in seq
    i = 0

    # swap : put a "copy of obj" in last place of seq
    # this copy is called "sentinel" - זקיף
    # it is possible that obj has more than one appearance in seq, and the last element is one of them

    last = seq[n]
    seq[n] = obj

    while seq[i] != obj:
        i += 1

    # restore original last element in the last position in seq
    seq[n] = last

    if i < n or seq[n] == obj: # the original last element can also be the obj
        return i
    else :
        return None
```

חיפוש בינארי

בסדרה ממויינת עם 2^{10} איברים אנחנו עושים לכל היותר 10 בדיקות.

הולכים לאמצע לאיבר ה- i, בודקים האם האיבר גדול יותר מהאיברים שרוצים או קטן יותר. אם גדול יותר, משליכים את החלק השמאלי ובודקים את האיבר האמצעי j בין i לבין הסוף. וממשיכים באותו אופן.

מקסימום מספר צווי החיפוש בסדרה ממוינת באורך n יהיה $\log_2 n$. הסיבוכיות: $O(\log n)$.

בחיפוש בינארי יותר נכון להשתמש בלולאה while.

```python
def binarySearch(seq, obj):
    left = 0             # left border
    right = len(seq)-1   # right border
    found = False

    while left <= right and not found:
        mid = (left + right)//2
        if seq[mid] == obj:  # mid is the location of the obj
            found = True
        else:
            if obj < seq[mid]:
                right = mid - 1
            else:
                left = mid + 1

    return found
```

סיבוכיות זמן ריצה

רוצים להביע את זמן הריצה כפונקציה של גודל הקלט.

גודל הקלט עצמו n חשוב רק אם הקוד מבצע משהו n פעמים.

Operations to consider for:

- arithmetic: addition, subtraction, multiplication, division,..
- comparisons
- assignment (binding variables)
- indexing into a container
- calling, returning from functions

לכל אופרציה בזאת יש "מחיר" שמשלמים ביעילות. כל פעולה בזאת לוקחת זמן קבוע ללא קשר לגודל הקלט. פעולות שונות לוקחות זמנים שונים. הזמנים גם שונים בין מחשבים, מערכות הפעלה ותוכנות.

מבלי להכנס לזמן שלוקחת כל פעולה נוכל לסמן פעולות שונות כך:

- c1 – arithmetic
- c2 – comparison
- c3 – assignment
- c4 – indexing
- c5 - call, return
- etc…

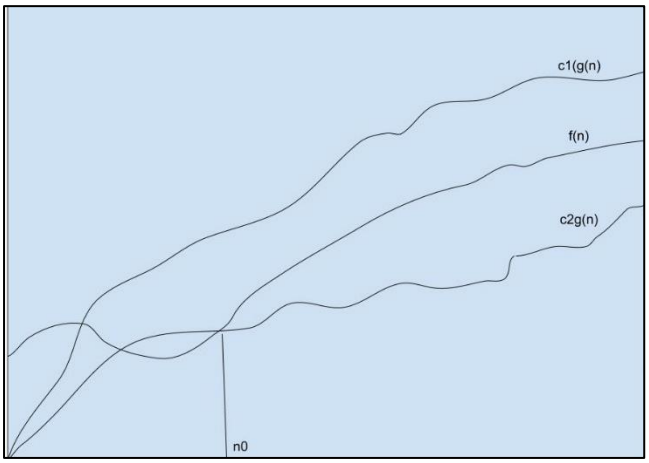
גבול תחתון (מהיר יותר).

גבול עליון (איטי יותר).

נאמר שזמן הריצה של קטע קוד הוא $\theta(n)$ כדי לציין ששני הגבולות, העליון והתחתון, של זמן הריצה הן פונקציות לינאריות של n. מתעלם מהקבועים ולכן מאבד דיוק אך מתרכז בגורם העיקרי של זמן הריצה ומאפשר השוואה בין אלגוריתמים שונים לפי האופן בו זמן הריצה מושפע משינויים בגודל הקלט (order of growth).

נוכל גם להכליל ולדבר על $\theta(g(n))$. לפונקציה נתונה $g(n)$ נוכל להגדיר את אוסף הפונקציות:

$$\theta\big(g(n)\big) = \{\ f(n)\ \mid\ \exists\ constants\ c_1, c_2, n_0 > 0\ \mid\ 0 \le c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)\ \ \forall\ n \ge n_0\ \}$$



בלומר, אוסף הפונקציות של $f(n)$ כך שקיימים קבועים c_1, c_2, n_0 גדולים כולם מ-0, כך ש- $f(n)$ נמצאת בין $c_1 \cdot g(n)$ לבין $c_2 \cdot g(n)$, לכל n שגדול מ- n_0.

התפקיד של n_0: לסמן החל מאיזה n יתקיימו האי שוויונות הללו.

התפקיד של c_1, c_2: להנמיך או להעלות את $g(n)$ מעל או מתחת לערכים של $f(n)$.

אם פונקציה $f(n)$ נמצאת באוסף הזה נוכל לומר ש- $f(n)$ היא $\theta(g(n))$.

linear_search

This algorithm stops when obj is found. we don't know in advance how many times the loop will iterate.

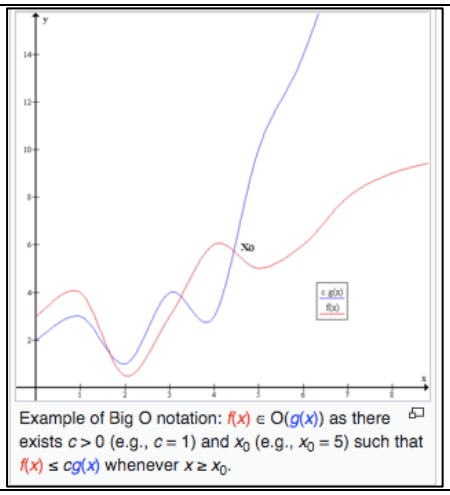
- Best case: one iteration - if obj is in seq[0], not dependent on `n`, we say $\Theta(1)$, which is a constant.
- Worst case: len(seq) iterations - if obj is not in seq, dependent on `n`, this case is $\Theta(n)$

In this example we'll use the Big-O notation:$O(n)$, to represent a function bounded from above by a function of `n` and from below by a `constant function`.

$$O\big(g(n)\big) = \{\ f(n)\ \mid\ \exists\ constants\ c, n_0 > 0\ \mid\ 0 \le f(n) \le c \cdot g(n)\ \ \forall\ n \ge n_0\ \}$$

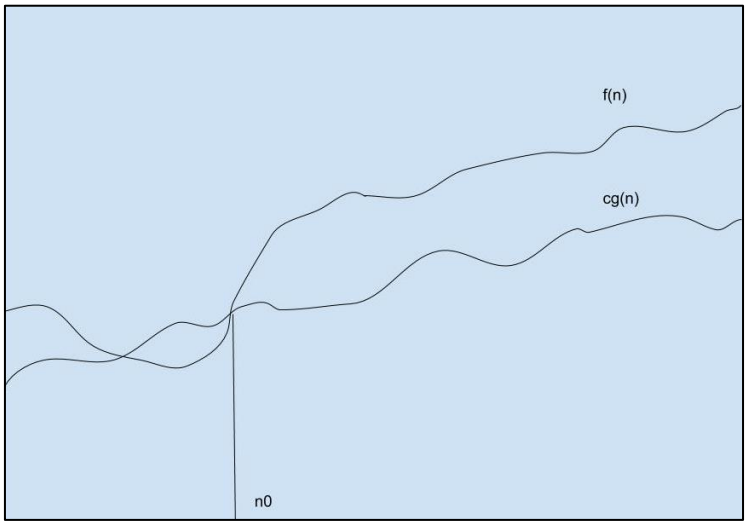
הפונקציה בדוגמה:

	Best case: $\theta(1)$

 Example of Big O notation: $f(x) \in O(g(x))$ as there exists $c > 0$ (e.g., $c = 1$) and x_0 (e.g., $x_0 = 5$) such that $f(x) \leq cg(x)$ whenever $x \geq x_0$.	*Worst case*: $\theta(n)$ *All cases*: $O(n)$

אם רוצים לציין שהגבול התחתון של פונקציה $f(n)$ הוא פונקציה $g(n)$, מבלי לציין גבול עליון של $f(n)$ נשתמש ב-

$$\Omega(g(n)) = \{\ f(n)\ |\ \exists\ constants\ c, n_0 > 0\ \ |\ \ 0 \leq c \cdot g(n) \leq f(n)\ \ \forall\ n \geq n_0\ \}$$



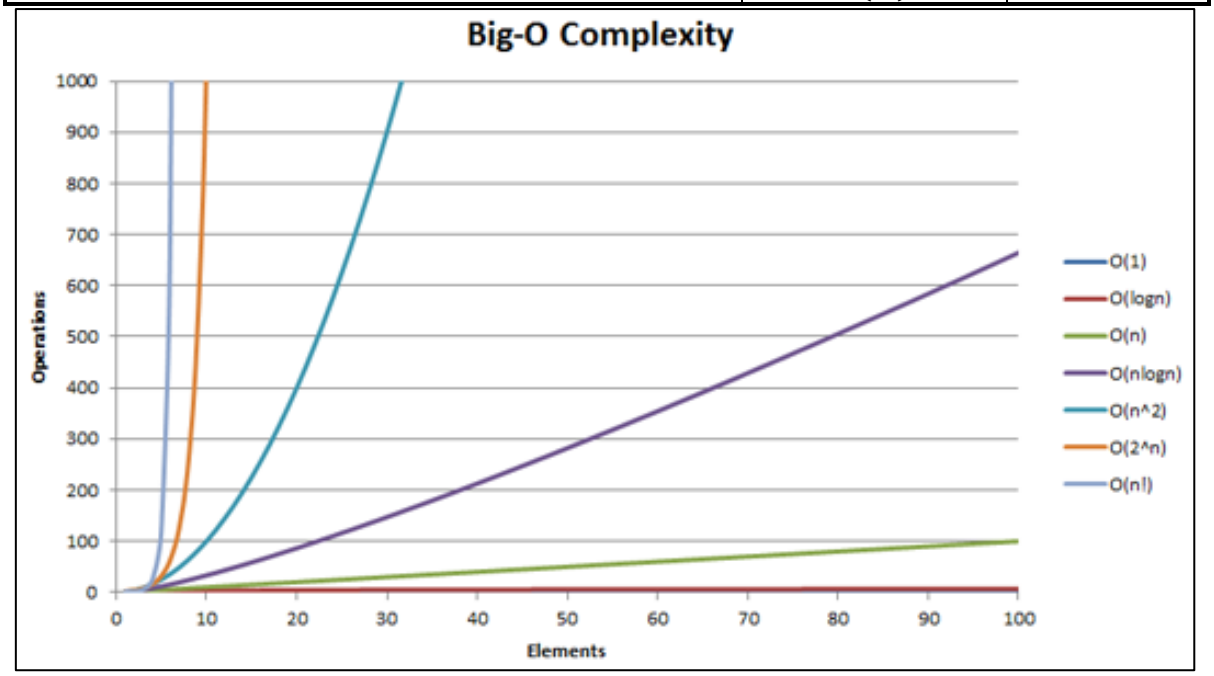
סימונים אסימפטוטיים: $\theta(n), \Omega(n), O(n)$. מתארים התנהגות של פונקציות כאשר *n* שואף לאינסוף.

After discarding all non dominant factors, what is left enable clear vision of the order of growth of the functions.

Common Big O running time values table

constant running time.	$O(1)$	*Good*
logarithmic running time. (any base we use is ok with the O() notation because they differ only by a multiplicative constant ,example: log2(a)=log10(a)*log10(2) and log10(2)=0.3010...)	$O(\log n)$	
square root running time	$O(\sqrt{n})$	
linear running time.	$O(n)$	*OK*
log-linear running time.	$O(n \log n)$	
polynomial running time. (k is a constant).	$O(n^k)$	*Bad*

exponential running time.	$O(c^n)$	*Awfull*
factorial running time	$O(n!)$	



Recitation 5

Runtime Analysis:

נספור כמה פעולות בסיסיות יש לנו בהינתן קלט בגודל n. כמה איטרציות/קריאות רקורסיביות, כמה אופרציות בכל אחת מהאיטרציות. תמיד נחשב את המקרה הכי גרוע כדי לחסום מלמעלה.

0 – חסם עליון.

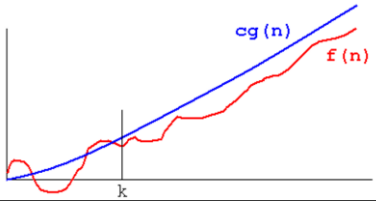
Big O notation:

פעולות שלא תלויות בגודל בקלט – זמן קבוע:

- Arithmetic operations (+, -, *, /, %)
- Comparison operators (<, >, ==, !=)
- Variable declaration
- Assignment statement
- Invoking a method or function

הגדרה:

- $f(n) = O(g(n))$ $(\exists C > 0, N_0 \ s.t. \forall n \geq N_0 : f(n) \leq C \cdot g(n))$
- *Basically, it tells you **how fast a function grows***



- ***Definition:** we say f(n) = O(g(n)) if*
  there exists:
  - a real number $C > 0$
  - a real number N_0

  such that **for all** $n \geq N_0$: $f(n) \leq C*g(n)$

במדידה עד כמה פונקציה גדלה מהר, לא איכפת לנו מכפל בקבוע, הוא זניח מבחינתנו. אנחנו מגדירים שפונקציות שנבדלות בכפל בקבוע חיובי הן אותו הדבר מבחינת החסם האסימפטוטי.

דוגמאות:

- Let $f(x) = 6x^4 - 2x^3 + 5$ and $g(x) = x^4$
- We will show a positive integer c and x_0 s.t $f(x) \leq c*g(x)$ for all $x \geq x_0$

Let $x_0 = 1$ and $c = 13$

$$f(x) = 6x^4 - 2x^3 + 5 \leq 6x^4 + 2x^3 + 5$$
$$\leq 6x^4 + 2x^4 + 5x^4$$
$$\leq 13x^4$$

$f(x) = O(x^4)$

- Define **n = len(lst)**
  - There are ***n*** iterations
  - In each we have **O(1)** operations

=> The runtime is ***O(n)***

```
def find_max(lst):
    max_num = 0
    for x in lst:
        if max_num < x:
            max_num = x
    return max_num
```

- Calculate $(4x-5)(2x^2+3x-6)$:
  - p1 = [4,-5]
  - p2 = [2,3,-6]

Naive implementation:

```
res = [0]*(len(p1)+len(p2)-1)
for ix1,item1 in enumerate(p1):
    for ix2,item2 in enumerate(p2):
        res[ix1+ix2] += item1*item2
```

הכפלת פולינומים:

בהשמה ב- res אנחנו שמים רשימה של אפסים באורך שהכפלנו בו.

Enumerate – בכל איטרציה באיבר הראשון (ix1) יהיה לי את האינדקס (p1[0]) ובאיבר השני (item1) יהיה לי את הערך (4).

Proof of correctness

כשאנחנו בונים אלגוריתם אנחנו צריכים להוכיח שהוא נכון – עושה את מה שאנחנו חושבים שהוא יעשה.

Loop invariant is a property that holds before (and after) each iteration. מאפיין שאנחנו טוענים שנשמר בכל אחת מהאיטרציות.

במקרה של bubble sort:

- The invariant is:

By the end of iteration i:

- ***the sub-array A[n-i-1:] is sorted***
- ***any element in A[n-i-1:] is ≥ any element in A[:n-i-1]***

הוכחה באינדוקציה

- **Base case proof:**
  - i = 0 , we bubble the largest item to the end of the list.

- **Induction step proof:**
  - **Assume**
    - **A[n-i-1:] is sorted**
    - **Any element *in A[n-i-1:] is ≥ any element in A[:n-i-1]***
  - **Iteration i+1 inserts at position n-i the largest of the remaining unsorted elements of A[:n-i-1]**, as computed by the inner loop.
  - Hence, by the end of iteration i+1
    - **A[:n-i] contains only elements smaller than A[n-i:]**
    - **and A[n-i] is smaller than any element in A[n-i:]**
  - **Thus A[n-i:] is sorted** ⇒ **invariant is preserved**

Termination:

after the last iteration (iteration # n-2):

- **A[1:]** is **sorted**
- all elements in **A[1:]** are >= elements in **A[0]**.

Therefore, **the list is sorted**

זמן ריצה של sort() ו- sorted() זה הזמן ריצה של האלגוריתם timesort בו פייתון משתמש למיון. הזמן ריצה הוא $O(n \log n)$.

- When **sorting** a list **without knowing anything on the elements values**, the best running time we will be able to achieve is **O(nlogn)**
- If we **have more information**, we may be able to **sort** the list faster (**O(n)**).

דוגמאות לאלגוריתמים בהם נוכל לקבל זמן ריצה יותר טוב מ- $O(n \log n)$ בהינתן קלט מתאים (עם מידע נוסף) : `counting sort, radix sort.`

למשל ב `counting sort` מיינים מספרים שלמים אי שליליים שחוזרים על עצמם. משתמשים באינדקסים של המערך.

כך הסיבוכיות יוצאת $O(n + k)$ כש- n גודל המערך המקורי ו- k הערך המקסימלי (איבר).

Lecture 6: Genexpressions, generators, memory considerations

אתר טוב בדי לראות מה קורה מאחורי הקלעים בזיכרון – python tutor. תוכנה – nbtutor (הדגמה ב7.1).

חידוד aliasing – זיכרון ויעילות

אנחנו לא יכולים לבטל את הaliasing אם המספרים שלנו הם בטווח של 5- עד 256. באופן דומה עם סטרינגים קטנים.

"because there are no 2 different numbers from this range (for יעילות reasons)"

```
# there is ONLY ONE object with value 256
a = 256
b = 256
print(id(a), id(b))
```

```
140707379274504 140707379274504
```

```
# OOPS ! we stepped out of the יעילות range - and got different ids
a = 257
b = 257
print(id(a), id(b))
```

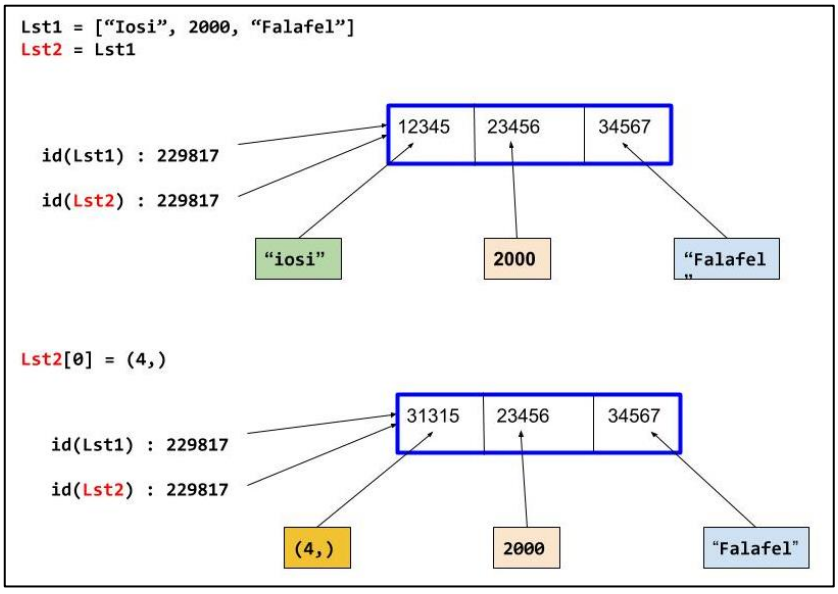
```
2902702501904 2902702503664
```

```
# aliasing with strings:
s1 = "Iosi"
s2 = s1
print(id(s1), id(s2))
```

```
2902703873200 2902703873200
```

```
# for small strings : the same "אותו יעילות"
# can not cancel the aliasing for short same-strings too
s2 = "Iosi"
print(id(s1), id(s2))
```

ברשימה יושבות הכתובות של האיברים בזיכרון. אם עושים השמה של איבר אחר הכתובת משתנה.



באשר יש לאותה רשימה 2 שמות, אם נוסיף איברים לרשימה דרך אחד השמות, הכתובת של הרשימה עצמה לא תשתנה. לעומת זאת, אם נשים שם רשימה אחרת הכתובת תשתנה. דוגמה להסבר הלא ברור:

``x = [1, 2, 3]
y = x
print(id(x), id(y))```
```2902703317696 2902703317696```	

``x += ['xyz'] # adds 'xyz' to list x
print(id(x), id(y))```
```2902703317696 2902703317696```
	`x += some list`

``# creates a new list [1,2,3,'xyz'] named x
x = x + ['abc']
print(id(x), id(y))```
```2902703880896 2902703317696```
	`x = x + some list`

זמן ריצה ועלויות בקונטיינרים שונים – הרבה תמונות

Running time

O(1) means constant time, not dependent on size of input *n*

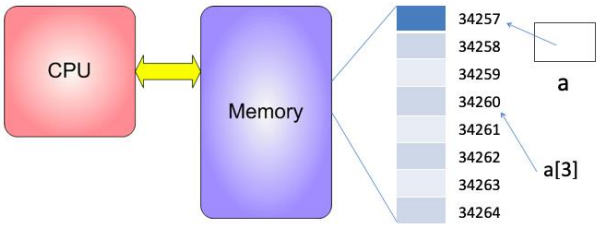
Operation	List / Tuple		Dict / Set	
Read data from key/index	O(1)		O(1)	
Update existing key/index	O(1)		O(1)	
Check membership	O(n)		O(1) for keys For values O(n)	
Add more items to the end	Worst case O(n)	Amortized O(1)	Worst case O(n)	Amortized O(1)
Insert an item	O(n)			
Delete an item				

See also:
https://wiki.python.org/moin/TimeComplexity

השקעה לאורך זמן

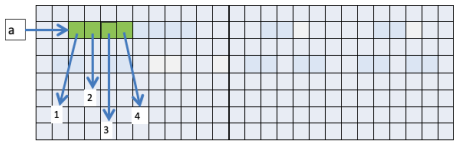
x

Random Access Memory



To access *a[i]*, Python takes a start location of *a*, and adds *i* to it to get the address in RAM

Python Implements Lists



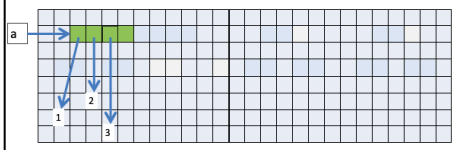
`a.append(4)`

`a` becomes `[1, 2, 3, 4]`

Here, append is O(1), free space was available

`len(a) == 4`

Python Implements Lists



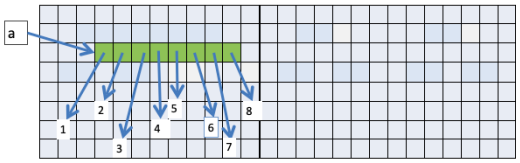
`a = [1, 2, 3]`

This results in Python setting aside, let's say, 4 places for the list a, but only using the first 3

`len(a) == 3`

(מימין לשמאל)

Python Implements Lists



We do three more O(1) appends, and

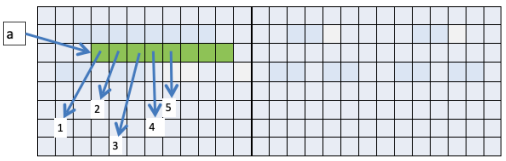
`a` becomes `[1, 2, 3, 4, 5, 6, 7, 8]`

`len(a) == 8`

Now do `a.append(9)`

No free space is available

Python Implements Lists



Now do `a.append(5)`

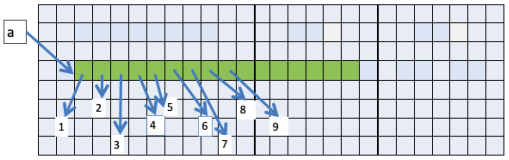
No free space. Python finds new (double) space for the list, updates `a`, copies pointers to new place

`a` becomes `[1, 2, 3, 4, 5]`

Here, append is O(n), no free space was available

`len(a) == 5`

Python Implements Lists



Python finds new (double) space for the list, updates a, copies pointers to new place

`a` becomes `[1, 2, 3, 4, 5, 6, 7, 8, 9]`

Here, append is again O(n), copying of list occurred

`len(a) == 9`

איך בדיוק עובד hashing:

נגיד שנרצה למצוא את הhashing value של מפתחות במילון. נתבונן בשארית החלוקה (למשל של 7) עם מודולו על הערכים שקיבלנו. השאריות שנקבל ישמשו אותנו כאינדקסים עבור הזוג של המפתח והערך שבדקנו. אם יש לנו שני מפתחות שונים עם אותו מודולו נאכסן את הזוג השני במקום פנוי אחר וכשנרצה להגיע אליו נוסיף עוד פעולה. אם הטבלה מספיק גדולה, שכיחות ההתנגשויות נמוכה.

עלויות של פעולות על רשימות:

Operation	Example	Complexity	Remarks
Index	`lst[i]`	`O(1)`	
Assign	`lst[i] = 2`	`O(1)`	
Length	`len(lst)`	`O(1)`	
Append	`lst.append(9)`	`O(1)`	amortized, worst: `O(N)`
Pop	`lst.pop()`	`O(1)`	same as lst.pop(-1)
Clear	`lst.clear()`	`O(1)`	similar to lst = []
Slice	`lst[a : b]`	`O(b-a)`	`lst[1:4]:O(1)` / `lst[:]: O(len(lst)-0)=O(N)`
Extend	`lst.extend(...)`	`O(len(...))`	
Construction	`list(...)`	`O(len(...))`	
check ==, !=	`lst1 == lst2`	`O(N)`	
Insert	`lst[a:b] = ...`	`O(N)`	also `lst[a:a]` = iterable
insert(method)	`lst.insert(i,ob)`	`O(N)`	
Delete	`del lst[i]`	`O(N)`	depends on `i` ; `O(N)` in worst case
Containment	`x in/not in lst`	`O(N)`	linearly searches list
Copy	`lst.copy()`	`O(N)`	Same as `lst[:]` which is `O(N)`
Remove	`lst.remove(...)`	`O(N)`	
Pop	`lst.pop(i)`	`O(N)`	`O(N-i): lst.pop(0):O(N)` (see above)
Min/Max	`min(lst)/max(lst)`	`O(N)`	linearly searches list for value
Reverse	`lst.reverse()`	`O(N)`	
Iteration	`for i in lst:`	`O(N)`	
Sort	`lst.sort()`	`O(N Log N)`	
Product	`lst*t`	`O(tN)`	

עלויות של פעולות על סטים:

Operation	Example	Complexity	Remarks
Length	`len(s)`	`O(1)`	
Add	`s.add(3)`	`O(1)`	
Containment	`x in/not in s`	`O(1)`	compare to list/tuple - O(N)
Remove	`s.remove(..)`	`O(1)`	compare to list/tuple - O(N)
Discard	`s.discard(..)`	`O(1)`	
Pop	`s.pop()`	`O(1)`	popped value "randomly" selected
Clear	`s.clear()`	`O(1)`	similar to s = set()
Construction	`set(...)`	`O(len(...))`	depends on length of ... iterable
check ==, !=	`s != t`	`O(len(s))`	same as len(t); False in O(1) if the lengths are different
<=/<	`s <= t`	`O(len(s))`	issubset
>=/>	`s >= t`	`O(len(t))`	is superset s <= t == t >= s
Union	`s | t`	`O(len(s)+len(t))`	
Intersection	`s & t`	`O(len(s)+len(t))`	
Difference	`s - t`	`O(len(s)+len(t))`	
Symmetric Diff	`s ^ t`	`O(len(s)+len(t))`	
Iteration	`for v in s:`	`O(N)`	Worst: no return/break in loop
Copy	`s.copy()`	`O(N)`	

<u>עלויות של פעולות על מילונים:</u>

Operation	Example	Complexity	Remarks
Index	`d[k]`	`O(1)`	
Store	`d[k] = v`	`O(1)`	
Length	`len(d)`	`O(1)`	
Delete	`del d[k]`	`O(1)`	
get/setdefault	`d.get(k)`	`O(1)`	
Pop	`d.pop(k)`	`O(1)`	
Pop item	`d.popitem()`	`O(1)`	popped item "randomly" selected
Clear	`d.clear()`	`O(1)`	similar to s = {} or = dict()
View	`d.keys()`	`O(1)`	same for d.values()
Construction	`dict(...)`	`O(len(...))`	depends # (key,value) 2-tuples
Iteration	`for k in d:`	`O(N)`	all forms: keys, values, items
			Worst: no return/break in loop

גנרטורים

פונקציות שיכולות ליצור נתונים בצורה מתוחכמת וגם להגיש אותם אחד אחד באמצעות איטרטור שהן מחזירות לשימושו של מזמין הנתונים.

Generator Expressions (genexps)

Create an iterator whose elements are constructed by the same techniques as list comprehensions.

כדי לקבל כל פעם את האיבר הבא צריך להפעיל את `next`.

```
GE = (x + '-' + y for x in 'iosi' for y in 'moshe' )
print('GE =', GE) # <generator object <genexpr> at 0x599c92c>
```

```
GE = <generator object <genexpr> at 0x000001FD94229C40>
```

```
# print(next(GE), end = ' ')
next(GE)
```

```
'i-m'
```

אם רוצים לקבל את כל האיברים בבת אחת אפשר להפעיל עליו `list` ואז להדפיס אותה.

```
GEList = list(GE)
print('GE elements in a list:', GEList)
```

גנרטורים – כללי

פונקציה	גנרטור
כאשר מזומנת נוצר namespace פרטי, בו נוצרים וחיים המשתנים. כשהפונקציה מסיימת ה- namespace	אפשר לשוב ולבצע את הפונקציה מהמקום בו השארנו אותה. לא זורקים את המשתנים. "פונקציה עם שלט"

מתבטל והמשתנים וערכיהם נמחקים. זימון אחר לפונקציה יוצר הכל מחדש.	
	סוג מסוים של פונקציה שמפשט את המשימה של יצירת איטרטורים.
	כשמזמנים גנרטור הוא לא מחזיר ערך בודד אלא אובייקט מטיפוס גנרטור שמשמש כאיטרטור.

בונים את הגנרטורים באמצעות `yield`:

```
def generate_ints(N):
    for i in range(N):
        yield i
```

```
g = generate_ints(213) # getting the שלט
```

```
next(g)
```

```
0
```

גנרטור יכול לזמן גנרטור אחר.

בעיות מיון

Bubble sort

<u>איטרציה 1</u>: משווים כל אלמנט לאלמנט שמימינו מ-0 עד n-1. אם הסדר לא נכון, כלומר שהאיבר הראשון גדול מהאיבר השני (בהנחה שמסדרים מהקטן לגדול) מחליפים ביניהם. בסוף האיטרציה האיבר הגדול ביותר ברשימה יגיע לסוף.

<u>איטרציה 2</u>: אותו דבר עד האיבר ה- n-2.

וכן הלאה עד שאין יותר אלמנטים להחליף וזה אומר שהמיון הסתיים.

```
def bubble_sort(lst):
    for i in range(len(lst)):
        swap = False
        for j in range(len(lst)-i-1):
            if(lst[j] > lst[j+1]):
                lst[j], lst[j+1] = lst[j+1], lst[j]
                swap = True
        if(not swap):
            break
    return lst
```

זמן ריצה של bubble sort:

במקרה הכי טוב	$O(n)$	▫ What would be the number of steps in the **best case**? (i.e., the input list is sorted) ▫ Start with the **first iteration**, going over all the list ▫ The list is sorted so there are **no swaps** ⇨ ***O(n) steps are required***

במקרה הכי גרוע	$O(n^2)$	▫ For a list of size *n*: ▫ the outer loop does n-1 iterations ▫ In each iteration *i* of the outer loop, the inner loop does *n-i-1* iterations: *(n-1) + (n-2) + … + 1 = n(n-1)/2* ▫ Note that each operation within the inner loop (comparing and swapping) takes **constant time** *c* ⇨ ***Number of steps is a polynomial of 2nd degree: O(n²)***

Selection Sort

הסדרה מחולקת לשני חלקים – חלק ממוין משמאל וחלק לא ממוין מימין. האלגוריתם בוחר את האיבר המינימלי בחלק הלא ממוין והוא מחליף מקום עם האיבר במקום השמאלי ביותר.

הסיבוביות של selection sort היא $O(n^2)$. מבוסס על 2 לולאות מקוננות כי קוראים לפונקציה הראשונה מתוך הפונקציה השניה.

```
def find_min(lst, start):
    min_ind = start
    for i in range(start+1, len(lst)):
        if lst[i] < lst[min_ind]:
            min_ind = i
    return min_ind

def selection_sort1(lst):
    for i in range(len(lst)-1):
        ind = find_min(lst, i)
        lst[i], lst[ind] = lst[ind], lst[i]
    return lst
```

21 27 14 **12** 40	12 is the smallest
12| 27 14 **21** 40	exchange positions 12 with 21 from 1st position
12| 27 **14** 21 40	14 is the next smallest
12 **14**| **27** 21 40	exchange positions 14 with 27 from 2nd position
12 14| 27 **21** 40	21 is the next smallest
12 14 **21**| **27** 40	exchange positions 21 with 27 from 3rd position

Insertion Sort

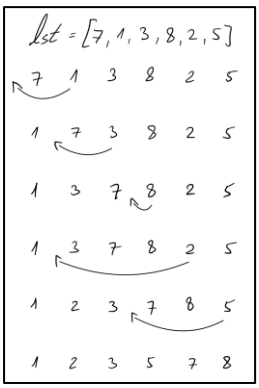
נתונה סדרה לא ממוינת. עוברים על האיברים שלה משמאל לימין. לפני כל צעד ימינה האלגוריתם דואג שהאיברים שנשארים מצד שמאל יהיו ממוינים. כשמגיעים לאיבר חדש שעדיין לא מוין, צריך למצוא את המקום שלו באזור שבבר סודר, אז מזיזים את האיברים שגדולים ממנו ימינה בבלוק. הסיבוביות: $O(n^2)$.

```
def insertionSort(lst):
    for slot in range(1, len(lst)):
        val = lst[slot]
        check_slot = slot - 1

        while check_slot >= 0 and lst[check_slot] > val:
            lst[check_slot + 1] = lst[check_slot]
            check_slot = check_slot - 1

        lst[check_slot + 1] = val

    return lst
```



Quick Sort

מיון בעזרת רקורסיה.

אלגוריתם הסתברותי והסיבוביות שלו היא בממוצע $O(n \log n)$. במקרה הכי גרוע $O(n^2)$ אבל זה נדיר.

הרעיון הכללי: בוחרים מיקום באופן אקראי. נשים את כל המספרים שקטנים מהמספר שבחרתי משמאלו, ואת כל המספרים שגדולים ממנו מימינו (בדוגמה הפונקציה `pertition`). עבשיו מיינים כל אחד מהצדדים באותו האופן.

```
def quicksort(data):
    quicksort_helper(data, 0, len(data))


def quicksort_helper(data, start, end):
    # start is the first index to sort
    # end is the index after the last one
    if(start < end-1):
        pivot_idx = partition(data, start, end)
        quicksort_helper(data, start, pivot_idx)
        quicksort_helper(data, pivot_idx+1, end)
```

```
def partition(data, start, end):  # assumes start<end
    # select a random pivot, place it in the last index
    pivot_idx = random.randint(start, end-1)
    pivot = data[pivot_idx]
    swap(data,pivot_idx, end-1)
    pivot_idx = end-1
    end -= 1

    while(start < end):
        if(data[start] < pivot):
            start += 1
        elif(data[end-1] >= pivot):
            end -= 1
        else:
            swap(data, start, end-1)
            start += 1
            end -= 1

    # swap pivot into place

    swap(data, pivot_idx, start)
    return start
def swap(data,ind1,ind2):
    data[ind1],data[ind2] = data[ind2],data[ind1]
```

Merge Sort

אלגוריתם למיון סדרות. הפונקציה מקבלת 2 סדרות ממוינות וממזגת אותן לסדרה אחת. הסיבוכיות: $O(n \log n)$.

The running time of merge sort for a sequence of length n is $T(n) = 2T\left(\frac{n}{2}\right) + n$ because it is equal to appling the algorithm on 2 sequences of size n/2 + the n steps to merge the resulting two lists.

```
def merge(b, c, a):
    """assume b, c are sorted and
       len(a) = len(b) + len(c), it merges lists
       b,c into a"""
    i, j, k = 0, 0, 0
    while k < len(a):
        if i < len(b) and (j >= len(c) or b[i] < c[j]):
            a[k] = b[i]
            i += 1
            k += 1
        else:
            a[k] = c[j]
            j += 1
            k += 1
```

Recursive Merge Sort:

```
def sort(a):
    """recursive sort"""
    if len(a) == 1:
        return
    mid = len(a)//2
    b = a[:mid]
    c = a[mid:]
    sort(b)
    sort(c)
    merge(b, c, a)
```

נוסף

Unpacking

Using * to collect in a list values whose bindings are not uniquely specified when unpacking:

```
x, y, *others = range(5)
x, y, others
```

```
(0, 1, [2, 3, 4])
```

```
others
```

```
[2, 3, 4]
```

```
a, *middle, c, d = range(5)
print(a, middle, c, d)
```

```
0 [1, 2] 3 4
```

```
*first, b, c, d = range(5)
first, b, c, d
```

```
([0, 1], 2, 3, 4)
```

בהרצאה 6.2 שווה להסתכל על התמונות שמתארות: how to multiply big numbers fast.

Recitation 6

רקורסיה - מבוא

דומה לאינדוקציה. הגדרה שמשתמשת בעצמה. לקחת בעיה מורכבת ולחלק לתתי בעיות, תתי מופעים. חלקים קטנים יותר של אותה בעיה עם ערכים פשוטים יותר. הגעה למקרה בסיס.

במדעי המחשב – פונקציה שקוראת לעצמה.

רקורסיה מורכבת מ-3 חלקים:

1. מקרה בסיס
2. צעידה לקראת מקרה הבסיס
3. האלגוריתם קורא לעצמו

מציאת GCD (מחלק מקסימלי משותף) ע"י שימוש ברקורסיה:

```
def gcd (x, y) :
    if y == 0:
        return x
    return gcd(y, x % y)
```

מציאת יחס הזהב:

$$1.6180339887\ldots = 1 + \frac{1}{\sqrt{5}} = \varphi = 1 + \frac{1}{\varphi}$$

כל פעם נציב את ה- φ הקודמת שקיבלנו במכנה. לא משנה איזו φ_0 נבחר, ברקורסיה נתקרב לערך האמיתי של יחס הזהב.

```
def gr_approx(n):
    if n <= 1:
        return 1
    return 1 + 1 / gr_approx(n - 1)
```

מקרה הבסיס - $n = 1$. יחשב לי את φ_n.

חישוב הגבול של $\sqrt{2 + a_{n-1}}$:

```
import math

def a(n):
    if n < 1:
        return 1
    return math.sqrt(2 + a(n - 1))
```

כל מה שאפשר לממש בלולאה אפשר לממש ברקורסיה ולהיפך. לולאות יותר יעילות מרקורסיה. לפעמים שימוש ברקורסיה הוא פשוט יותר משימוש בלולאה.

יש מגבלה בזיכרון של אורך הרקורסיה כי כל קריאה לפונקציה פותחת עבורה namespace בזיכרון. מגבלה על כמות הפעמים שאפשר להכנס פנימה.

חידודים

- $x = \log_2 n$ זה כמות הפעמים שאני צריכה להעלות את 2 בחזקת x כדי להגיע ל- n.
- פרקטל: צורה שמוגדרת ע"י עצמה.

Lecture 7: Recursion; Backtracking

אין ברירה צריך לראות את ההרצאה והתרגול, מבינים דרך הדוגמאות.

רקורסיה

פונקציה רקורסיבית היא פונקציה שמזמנת את עצמה.

An inductive definition of a set L:

Describes the elements in the set L in terms of other elements in the set L.

Example: a definition of the set N of natural numbers is:

1. 1 is in N.
2. If an element x is in N then x+1 is in N.

N is the intersection of all sets satisfying (1) and (2).

A recursive definition of a function f(..):

Defines values of f(..) for some inputs, in terms of the values of the same function f(..) for other inputs.

Example, the rules to define the factorial n!:

1. 0! = 1
2. (n + 1)! = (n + 1) * n!

These rules cover all n >= 0, and the recursion gets to the base case n == 0.

These rules provide a construction formalism, used when writing the function that implements the factorial for inputs starting from 0, and going up.

```
def fact(n):
    if n == 0:          # the base case
        return 1
    return n * fact(n - 1)          # the reduction case
```

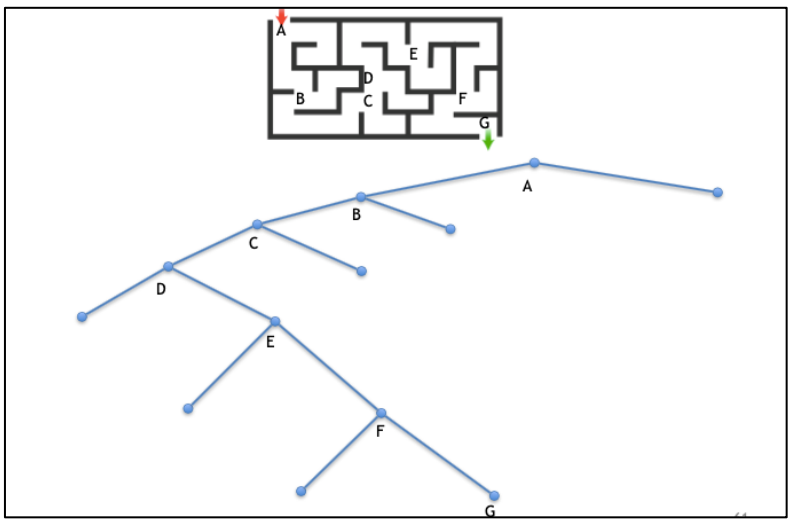
רצוי שלפונקציה רקורסיבית יהיו 1 או יותר מקרי בסיס שמחזירים ערכים ללא זימונים רקורסיביים נוספים – כדי לעצור את שרשרת הזימונים הרקורסיביים.

Backtracking

טכניקה אלגוריתמית רקורסיבית, לרוב לפתרון בעיות חישוביות עם אילוצים.

הרעיון הכללי הוא לבנות פתרון חלקי, להרחיב אותו, ואם מגיעים למבוי סתום לסגת לשלב מוקדם יותר ולנסות אופציה אחרת.

למשל – מעבר במבוך:



אלגוריתם מגניב שמחזיר את כל הדרכים האפשריות לעבור את המבוך מנקודה a לנקודה b:

```
maze=[["", "W", "", "", ""],
      ["", "W", "", "W", ""],
      ["", "W", "", "W", ""],
      ["", "" , "", "" , ""],
      ["", "W", "", "W",""]]
```

W מייצג קיר.

```
MOVES = {"u":(-1,0),"d":(1,0),"l":(0,-1),"r":(0,1)}

def print_maze_and_path(maze,path):
    print("\n\n" + str(path) + "\n")
    for line in maze:
        for cell in line:
            print(str(cell).center(3),end = " ")
        print()
def do_move(place,change):
    return (place[0]+change[0], place[1]+change[1])

def can_travel(point,maze):# can go to point ?
    return point[0]>=0 and point[0]<len(maze) and point[1]>=0 and \
           point[1]<len(maze[point[0]]) and not maze[point[0]][point[1]] # empty str/slot
```

```
def print_all_paths(start,goal,maze):
    _print_all_paths_helper(start, goal, maze, [])

def _print_all_paths_helper(start, goal, maze, path):
    if not can_travel(start,maze):
        return

    maze[start[0]][start[1]] = len(path)+1

    if start == goal:
        print_maze_and_path(maze,path)

    else:
        for direction in MOVES:
            path.append(direction)
            new_start=do_move(start, MOVES[direction])
            _print_all_paths_helper(new_start, goal,maze, path)
            path.pop()
    maze[start[0]][start[1]] = ""
```

```
print_all_paths((0,0), (4,4),maze)
```

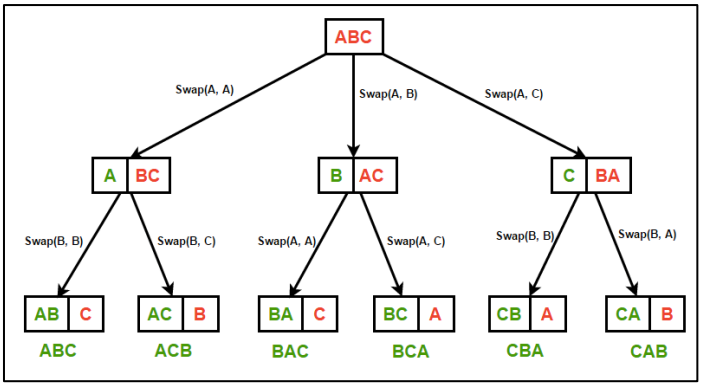
```
['d', 'd', 'd', 'r', 'r', 'u', 'u', 'u', 'r', 'r', 'd', 'd', 'd', 'd']

 1   W   9   10  11
 2   W   8   W   12
 3   W   7   W   13
 4   5   6       14
     W       W   15


['d', 'd', 'd', 'r', 'r', 'r', 'r', 'd']

 1   W
 2   W       W
 3   W       W
 4   5   6   7   8
     W       W   9
```

פרמוטציות של אברי רשימה - $n!$ אפשרויות: (האלגוריתם המלא בהרצאה 7.2)



מציאת מסלול של פרש שממלא עם תנועותיו את כל לוח השח:

1	58	39	48	51	56	41	46
38	49	2	57	40	47	52	55
59	28	31	50	3	54	45	42
30	37	12	27	32	43	4	53
11	60	29	36	13	26	33	44
20	63	8	17	24	35	14	5
61	10	19	22	7	16	25	34
64	21	62	9	18	23	6	15

נוסף

עיצוב אלגוריתמים: טכניקה אלגוריתמים בסיסית: הפרד ומשול. כדי לפתור בעיה יש לחלק אותה לתת בעיות קטנות יותר. למשל, ברקורסיה.

Stack overflow: מצב בו המחסנית בה שומרים את זימוני הפונקציות מתמלאת. למשל כאשר לא כותבים מקרה בסיס שיעצור את הזימונים הרקורסיביים.

(*בתרגול 7 יש הרבה דוגמאות חשובות ל- backtracking*)

Lecture 8: Object Oriented Programming [OOP]

תכנות מונחה עצמים

בתכנות מונחה עצמים מארגנים ביחד נתונים בפונקציות לאובייקטים. ואז ניתן לתאר פתרון של בעיה חישובית כסדרה של אינטראקציות בין האובייקטים.

בפייתון יש אובייקטים מובנים מראש. למשל – רשימות. השיטות append ו- sort שייכות ל- class list.

מחלקה (class) היא מכאניזם שמשמש ליצירת סוגים חדשים של אובייקטים. מגדירים אותה באופן הבא:

```
class ClassName:
    <statement-1>
    .
    .
    .
    <statement-N>
```

בדר"כ המחלקה היא אוסף של משתנים ופונקציות שנקראות שיטות. לרשימת הארגומנטים של השיטות הללו יש הרכב שנובע מאופן הזימונים שלהם.

מרחב השמות (namespace) – לכל מחלקה עם הגדרתה נוצר מרחב שמות אשר מהווה טווח לוקאלי בו מוכרים שמות המשתנים והפונקציות שנמצאות בפנים.

בדומה לפונקציות כאשר מחלקה מוגדרת נוצר אובייקט מסוג type (בפונקציות נוצר אובייקט מסוג function). עם האובייקט הזה אפשר לפנות לאלמנטים של המחלקה (משתנים או פונקציות) וליצור אובייקטים חדשים מהטיפוס של המחלקה.

מוסכמה: שמות של מחלקות חדשים מתחילים באות גדולה.

מילה מוסכמת: self. מתייחס לאובייקט עצמו שנוצר ומשתמש בשיטה.

לכל שיטה במחלקה תהיה שיטה f והארגומנט הראשון מתייחס לאובייקט עצמו. Self.

משתנים במחלקה נקראים "משתנה מחלקה" והם משותפים לכל האובייקטים שיווצרו במחלקה.

הגדרת משתנה פרטי למחלקה: `__varname`

אתחול אובייקט עם ערכים מסוימים: `__init__()`

```
class MyClass3:
    def __init__(self):
        self.data = [] # data is now a list, any object of type MyClass, can put some personal data in the list
        print("self id", id(self))
```

When a class defines an `__init__()` method, class instantiation automatically invokes the method `__init__()` for the newly-created object (class instance).

כל מחלקה יורשת תכונות מהמחלקה המקורית של פייתון.

How to dynamically add variables to the class namespace, and to the objects namespaces from outside of these namespaces:

Let's add `classVar` - a class variable, to the class SomeClass

```
SomeClass.classVar = 10 # that's it !
```

```
SomeClass.classVar
```

```
10
```

Let's add `instVar` - an instance variable to a specific object of class SomeClass

```
z = SomeClass() # create z, a new object of type SomeClass
z.instVar = 12 # we add instVar to object z (an instance variable)
z.instVar
```

```
12
```

`instVar` is an instance variable of `z`

```
# lets look:
SomeClass.__dict__
```

```
mappingproxy({'__module__': '__main__',
              '__doc__': ' An empty class ?',
              '__dict__': <attribute '__dict__' of 'SomeClass' objects>,
              '__weakref__': <attribute '__weakref__' of 'SomeClass' objects>,
              'classVar': 10})
```

```
z.__dict__
```

```
{'instVar': 12}
```

```
# Can we delete them ?
del SomeClass.classVar
SomeClass.__dict__
```

```
mappingproxy({'__module__': '__main__',
              '__doc__': ' An empty class ?',
              '__dict__': <attribute '__dict__' of 'SomeClass' objects>,
              '__weakref__': <attribute '__weakref__' of 'SomeClass' objects>})
```

```
del z.instVar
z.__dict__
```

```
{}
```

הוספת אובייקט פרטי למשתנים:

```
class Point:
    x = -1

p1 = Point()
p2 = Point()
```

```
# adding an instvar x to object p1 namespace, and binding it to 10
p1.x = 10
```

```
print("p1 is using its own var x: ", p1.x)     # p1 uses its (new) var x value
print("p2 is using Point.x", p2.x)     # p2 uses Point.x
```

```
p1 is using its own var x:  10
p2 is using Point.x -1
```

```
print(p2.__dict__)
```

```
{}
```

```
print(p1.__dict__)
```

```
{'x': 10}
```

דוגמאות:

הטלת מטבע:

```
from random import choice

class Coin0:
    faces = (1, 2) # class var  - 2 faces coin

    def toss(self):
        self.face_now = choice(Coin0.faces)
        return self.face_now

c1 = Coin0() # 1st object of class Coin0
c2 = Coin0() # 2nd object of class Coin0
```

```
c1.toss()
```

```
2
```

עם שמות השחקנים:

```
from random import choice
class Coin2: # my pocket casino
    """obj methods:toss()"""
    faces = (1, 2) # class var
    players = 0

    def __init__(self, fname): # initializer
        self.face_now = 0    # object var
        self.name = fname    # player name
        Coin2.players += 1
        print(fname, "is player number:", Coin2.players)

    def toss(self):
        self.face_now = choice(Coin2.faces)
        return self.face_now

jo = Coin2('Johnny')
mo = Coin2('Mony')
```

```
Johnny is player number: 1
Mony is player number: 2
```

```
# Action:
jo.toss()
mo.toss()

print('jo toss: ', jo.face_now)
print('mo toss: ', mo.face_now)
print("N of players: ", Coin2.players)
```

```
jo toss:  1
mo toss:  1
N of players:  2
```

דוגמאות נוספות (ומומלצות) ב- 8.1B,C.

בתכנות מונחה עצמים קיימים 3 סוגי **הגנה על אטריביוטים** של מחלקה. הם המשתנים והשיטות:

- `Name`
  - Public, can be freely used inside or outside of a class definition.
- `_name`

    - Protected, should not be used outside of the class definition, unless inside of a subclass definition, but this sometimes looks as a recommendation
- `__name`
    - Private, inaccessible and invisible, not possible to read/write to those attributes, except inside of the class definition itself.
        - לא באמת באמת מוגן אלא אפשר לגשת אליו אם פונים אליו "בשם המלא" – `my_pokemon._Pocemon__level -= 10` (אבל לא לעשות את זה).

Scoping Rules:

Classes define a namespace, but do not create a scope for the names inside the methods.

This is why, references to attributes and methods must be fully qualified: `self.balance` instead of `balance`.

The same when calling a method from another.

```python
class A():
    def b(self):
        print("b!")

    def s(self):
        #b(self)    # raises a Name error
        #b()        # raises a Name error
        self.b()    # ok
        A.b(self)   # ok
```

Encapsulation

Access attributes only thru specially defined methods.

By providing two kinds of methods for attributes:

- The **getter** methods: for accessing the values of attributes, without changing them. קריאת ערכים בלי לשנות אותם.
- The **setter** methods: used for changing the values of attributes. שינוי ערכים בצורה מבוקרת.

נשתמש במילים `set` ו- `get`.

```python
class Account4:
    adoc = "great bank"
    acc_number = 0
    num_of_accounts = 0

    def __init__(self, name):
        self.name = name
        Account4.acc_number += 1
        Account4.num_of_accounts += 1

    def set_balance(self, balance):
        self.balance = balance
        if not balance:
            self.balance = 0

    def get_balance(self):
        return self.balance
```

Inheritance

ירושה – יצירת מחלקות חדשות ממחלקות קיימות.

The existing classes are named **super** or **base classes**, and the derived classes: **sub-classes**.

Inheritance enables building classes based on attributes of existing classes.

An object of a subclass, created through inheritance, inherits all the attributes of the base class (except some methods like: constructors, destructors,..)

This allows code reusing.

Any new attributes (instances variables and methods) can be added to the sub-classes.

```
class Animal:
    def __init__(self, kind, voice):
        self.kind = kind
        self.voice = voice

    def get_kind(self):
        return self.kind

    def get_voice(self):
        return self.voice


class Fish(Animal):
    def swim(self):  # Fish method
        return self.voice


class Bird(Animal):
    def can_fly(self): # Bird method
        if (self.kind == 'chicken') or (self.kind == 'hodu') or (self.kind == 'parrot'):
            return False
        return True


class Speaker(Bird):
    def say(self):
        return('Elvis is comming !')
```

נוסף

פונקציה מובנית של פייתון: `getattr()` – מחפשת בין האטריביוטים של פונקציה או מחלקה (אם יש להן), אטריביוט בשם מסוים. אם אין, מחזירה ערך דיפולטי.

```
def f1(x):
    f1.count = getattr(f1, "count", 0) + 1
    return -89

for k in range(5):
    f1(k)

print(f1.count) # count is an attribute added to the name: f1
print(f1.__dict__)
```

```
5
{'count': 5}
```

Recitation 8

עד עכשיו למדנו POP – Procedural Oriented Programming.

עבשיו OOP – Object Oriented Programming.

האובייקטים ב- OOP משלבים בתוכם סוגי דאטא (מאפיינים) ופונקציות (מתודות).

While `class` is the blueprint, an `object` is an `instance` of the class with actual values.

For example, a Pokémon named 'Pikachu' of type 'Electric'.

Put it another way - a `class` is like a template to define the needed information, and an `object` is one specific copy that filled in the template.

Objects created from the same class are **independent** from each other.

The properties that a class defines are called `data members` or `member variables`.

These members can have different values for different objects.

Accessing members of an object is done in the following way: `object.member`.

בשונה מפונקציות רגילות מתודות תמיד קשורות למחלקה. אי אפשר להפעיל אותן מחות למחלקה.
`object.method()`.

`self` – בארגומנט הראשון נכנס האובייקט שעליו אנחנו פועלים. אז "משאירים לו מקום". קוראים למתודות מאובייקטים ספציפיים.

`member` זה בעצם המאפיינים שבתוך המתודות (name, number, super_power,...).

`__init__()` : רצה בעת יצירת האובייקט. במתודה הזאת קובעים מה יקרה כשיוצרים אובייקט. למשל, כשיוצרים פוקימון חדש כבר יווצר מידע, המתודה תצפה לקבל כבר מהתחלה את המאפיינים עליו. "שדות חובה". חייב להכניס את הארגומנטים.

בנוסף, אפשר להכניס כבר ערכים דיפולטיביים לכל אחד מהארגומנטים.

בברירת מחדל יש מתודת `init` שנוצרת ולא עושה כלום.

<u>משתנה מחלקה</u>

מחלקות יכולות גם להכיל משתנים. לכל אחד מהאובייקטים יש גישה אליהם. אם נשנה אותם באמצעות הם ישתנו בכל האובייקטים. בשביל להגדיר משתנה מחלקה כותבים אותו בלי `self.` לפני.

אפשר לראות ולשנות משתני מחלקה על ידי כתיבת שם המחלקה לפני:

```
✓ Pokemon.num_pokemons
✓ Pokemon.num_pokemons = 899
```

אפשר לגשת למשתנים דרך אובייקט ספציפי אבל אפשר רק לראות אותם. אם נשנה דרך האובייקט הספציפי הוא ישנה רק לאותו האובייקט ואנחנו דורסים את הגישה של אותו אובייקט למשתנה המחלקה הזה:

```
✓ my_pokemon.num_pokemons
✗ my_pokemon.num_pokemons = 899
```

<u>אנקפסולציה:</u>

מניעת גישה. צימוד דאטא ופונקציות / הפיכת דברים לפרטיים.

- מניעת באגים

- אפשר להחביא מימושים של דברים מסוימים. application programming interface – API. ממשק תוכנה יישום. הדרך שבה אפשר לתקשר עם קוד אחר. הקודים בבלל לא צריכים להכיר אחד את השני אבל יכולים להשתמש במתודות אחד של השני. כל קוד, מחלקה, יחידה עצמאית.

Getters and Setters:

שינוי ערכים מוסתרים (__name) מתודות שהמטרה שלהן היא לקבל את הערך המוסתר או לשנות אותו.

```
class Pokemon:
  # some class variables
  def __init__(self, name, pokemon_type,
  level):
    self.__name = name
    self .__type = pokemon_type
    self .__level = level

  def get_level(self):
    return self.__level

  def set_level(self, new_level):
    self.__level = new_level
```

*אפשר לעשות גם מתודות חבויות, כמו עם אטריביוטים, גם כאן עם __.

מתודות ברירת מחדל:

כשיוצרים מחלקה נוצרות מתודות נוספות של פייתון. אפשר לראות אותן עם הפעולה dir(). אפשר לדרוס ולשנות אותן. למשל:

`print(obj)` ⇔ `print(obj.__str__())` ⇔ `print(str(obj))`

כדי לדרוס את זה ניצור פונקציה באותו השם:

```
class Pokemon:
  # class implementation
  def __str__(self):
      return self.__name
```

ואז נקבל

```
print(my_pokemon)
>> Pikachu
```

Represent - כשרושמים את האובייקט באינטרפרטר. נשתמש בזה בדר"כ בדיבאגינג. כשסתם כותבים את השם באינטרפרטר מחזיר את הid.

```
class Pokemon:

    def __init__(self, name, pokemon_type, level):
        self.name = name
        self.type = pokemon_type
        self.level = level

    def __str__(self):
        return self.name

    def __repr__(self):
        return f'id={id(self)}, name={self.name}, type={self.type}, level={self.level}'


my_pokemon = Pokemon('Pikachu', ['Electric'], 12)
```

```
In[3]: my_pokemon
Out[3]: id=2375620451104, name=Pikachu, type=['Electric'], level=12
In[4]: print(my_pokemon)
Pikachu
```

Contains – בודק אם אלמנט נמצא באובייקט.

`element in obj ⇔ obj.__contains__(element)`

```
class PokemonTrainer:
    # class implementation
  def __contains__(self, pokemon):
      return pokemon in self.pokemons
```

מתודות נוספות:

+	object.__add__(self, other)
-	object.__sub__(self, other)
*	object.__mul__(self, other)
//	object.__floordiv__(self, other)
/	object.__div__(self, other)
%	object.__mod__(self, other)
**	object.__pow__(self, other[, modulo])
<<	object.__lshift__(self, other)
>>	object.__rshift__(self, other)
&	object.__and__(self, other)
^	object.__xor__(self, other)
|	object.__or__(self, other)
<	object.__lt__(self, other)
<=	object.__le__(self, other)
==	object.__eq__(self, other)
!=	object.__ne__(self, other)
>=	object.__ge__(self, other)
>	object.__gt__(self, other)

Lecture 9: More OOP; Linked data structures: Lists

Stack

מחסנית – Stack

מבנה נתונים מופשט אשר משמש כקונטיינר מסודר לאובייקטים. יש לו 2 פעולות עיקריות:

1. `push(item)` – להוספת אובייקט לקונטיינר
2. `pop()` – לסילוק האובייקט שהתווסף אחרון לקונטיינר ועדיין נמצא בו

Last In First Out – אחרון שנכנס יצא ראשון.

הקצה שבו הפעולות מתבצעות נקרא ראש המחסנית.

בפעולת Push מציינים איזה אובייקט ייכנס למחסנית ואילו בפעולה ה- Pop לא מציינים, פשוט מסלקים את האיבר שבראש המחסנית.

נוסיף עוד פעולה שתבדוק האם המחסנית ריקה: `is_empty()`.

ברשימות יש pop שאפשר להגיד לו איזה איבר באיזה אינדקס להוציא (`list.pop([i])` . אם לא רושמים דבר pop מוציאה את האיבר האחרון ברשימה.

```
class Stack1():

    def __init__(self):
        self.__lst = [] # items are pushed in - poped out of this list

    def push(self, item):
        self.__lst.append(item)

    def pop(self):
        return self.__lst.pop()

    def is_empty(self):
        return len(self.__lst) == 0

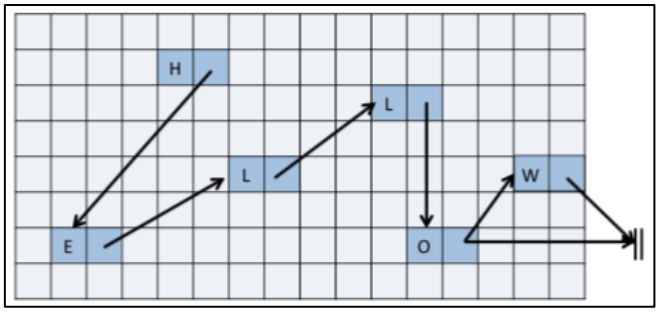
    def show(self): # not in stack repertoire, here for ilustration purposes
        return self.__lst
```

בפייתון הרשימות ממומשות בבלוקים רציפים בזיכרון. אם צריכים להגדיל את גודל הבלוק בגלל פעולת הוספה כלשהי צריך לחפש מקום גדול יותר ולהעתיק לשם את התוכן. למשל רוצים להוסיף עוד איבר אחד וחסר מקום אז מכפילים את הבלוק.

Linked Lists (Node)

רשימות משורשרות:

הרעיון הוא להשתמש בבלוק בעל 2 מקומות (שדות) עבור כל אובייקט מהרשימה. בשדה הראשון יהיה האובייקט (לינק אליו) כמו ברשימות הרגילות. בשדה השני, קישורית (לינק) לבלוק הבא. שרשרת של בלוקים בעלי 2 שדות כל אחד.



אפשר לממש זאת באמצעות המחלקה ()Node – היחידה הבסיסית של רשימות משורשרות.

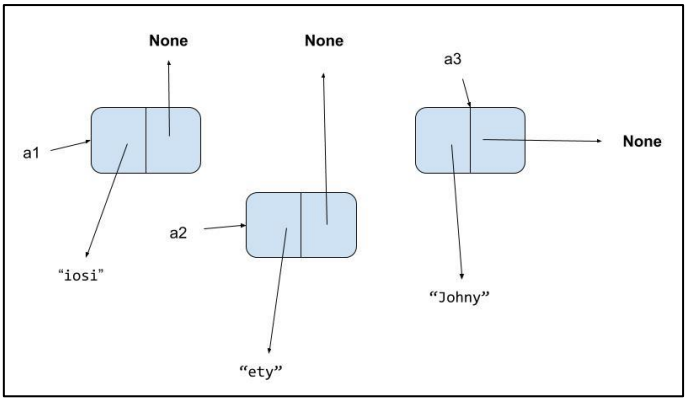
האיבר של הרשימה יכנס למשתנה data. כתובת הבלוק הבא ברשימה יכנס ל- next.

```
# a single basic Node class
# with it we can build complex data structures

class Node0:
    def __init__(self, data = None, next = None):
        self.data = data
        self.next = next
```

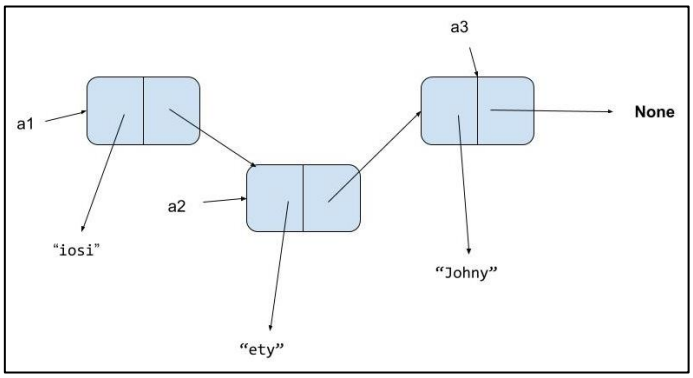
יצירת הבלוק:

```
a1 = Node0('iosi')
a2 = Node0('ety')
a3 = Node0('Johny')
```



קישור הבלוק הבא:

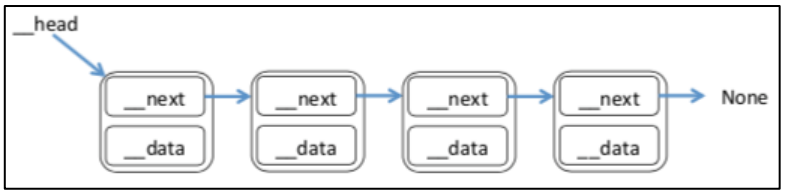
```
a1.next = a2
a2.next = a3
```



יצירת הרשימה המשורשרת תחת שם אחד:

```
# more compact writing
b1 = Node0('iosi', Node0('ety', Node0('johny')))
```

גישה לנתונים ברשימה:



נשתמש במשתנה עזר (aliasing) אשר מתקדם לאורך הרשימה, שבכל צעד יצביע על הnode הבא ברשימה המשורשרת:

```
curr = head
while curr != None:
    print(curr.data)
    curr = curr.next
```

```
# assume head is the 1st ekement of a list
# curr becomes alias of head of list
curr = head

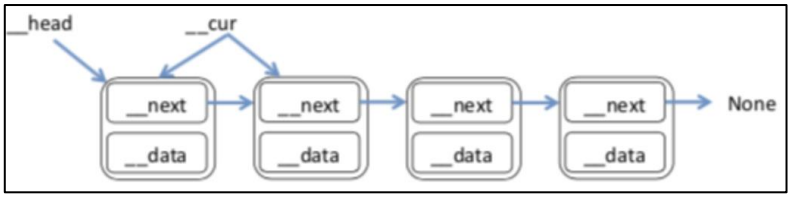
# curr becomes alias of 2nd node of list
curr = curr.next

# curr becomes alias of 3rd node of list
curr = curr.next

#... and so on
```

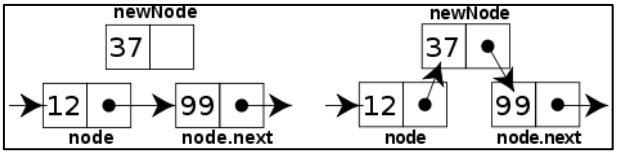
```
__cur = __head

__cur = __cur.__next
```



הכנסת איבר למקום נתון מראש ברשימה:

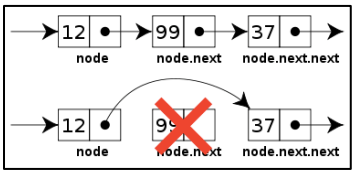
נעדכן שה- next של הבלוק החדש יצביע לבלוק הבא במקום שרוצים להכניס אותו (בשלב הזה יהיו שני מצביעים לאותו בלוק) ואז נשנה את ה- next של הבלוק הקודם שיצביע על הבלוק החדש.



להיזהר לא למחוק את כל המצביעים על בלוק מסוים כי אז הוא יכול להיאבד.

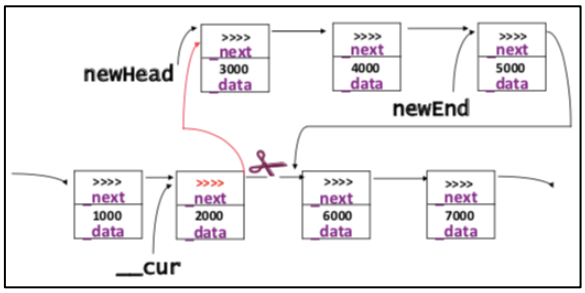
הסרת איבר מרשימה:

משנים את ה- next של האיבר הקודם ככה שיצביע לאיבר שאחרי האיבר אותו אנחנו רוצים להסיר. נשאר בזיכרון ללא יכולת להגיע אליו. `curr.next = curr.next.next` .



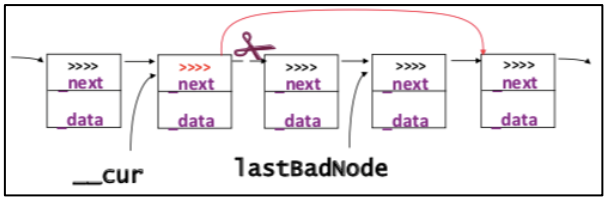
על מנת להסיר את האיבר הראשון נשתמש בפקודה: `head=head.next` .

הכנסת תת רשימה נתונה לתוך רשימה אחרת במיקום נתון:




```
newEnd.next = curr.next
curr.next = newHead
```

באותו אופן נמחק תת רשימה:




```
cur.next = lastBadNode.next
```

רשימה מעגלית:

מסמנים עם current את האיבר האחרון ונפנה אותו ל- head:

```
# setting curr to point to the last member here
curr = head.next.next

curr.next = head
```

סיבוכיות פעולות על רשימות מול רשימות משורשרות:

Operation	Linked List	Python's list
Insert at head	$O(1)$	$O(n)$
Insert at middle	$O(n)$, $O(1)$ given a reference	$O(n)$
Remove from head	$O(1)$	$O(n)$
Remove from middle	$O(n)$, $O(1)$ given a reference	$O(n)$
Find k-th element	$O(k)$	$O(1)$
Search unsorted	$O(n)$	$O(n)$
Search sorted	$O(n)$	$O(\log n)$

בעיות עם רשימות משורשרות:

- זמן:
  - כדי להגיע לאיבר צריך לעבור על כל האיברים שבדרך
  - הנתונים לא מאוכסנים ברצף. אחרי כל נתון יש את הכתובת של ה- node הבא.
- זיכרון:
  - לכל שדה נתונים, משתנה, יש "ליווי" של המצביע. עוד זיכרון.
  - בעייתי ללכת אחורנית. פתרון אפשרי – רשימות משורשרות כפולות. כל כל נתון יש שני מצביעים. אחד מצביע קדימה אל האיבר הבא ואחד אחורה לאיבר הקודם.

יצירת רשימה משורשרת כפולה:

```
class Doubl_Node:
    def __init__(self, data = None, next = None, prev = None):
        self.__data = data
        self.__next = next
        self.__prev = prev
```

Class Stack באשר המחסנית ממומשת על ידי רשימה משורשרת:

```python
class Node:
    def __init__(self, data = None, next = None):
        self.__data = data
        self.__next  = next

    # 2 getters
    def get_data(self):
        return self.__data

    def get_next(self):
        return self.__next

    # 2 setters
    def set_data(self, data):
        self.__data = data

    def set_next(self, next):
        self.__next = next


class Stack:
    def __init__(self, head = None):
        self.__head = head

    def push(self, new_head):
        new_head.set_next(self.__head)
        self.__head = new_head

    def pop(self):
        result = self.__head.get_data()
        self.__head = self.__head.get_next()
        return result

    def is_empty(self):
        return self.__head == None
```

מחלקה של רשימה משורשרת – **linkedlist class**:

```python
class LinkedList:
    def __init__(self, head = None):
        self.__head = head

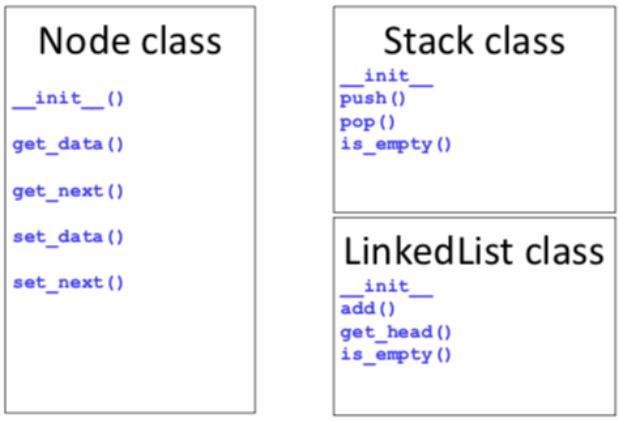
    def add(self, new_head):
        new_head.set_next(self.__head)
        self.__head = new_head

    def get_head(self):
        return self.__head

    def is_empty(self):
        return self.__head == None
```

Add – דומה לפוש של מחסנית. מקבלת אובייקט חדש מטיפוס node (new_head) ומעדכנת את השדה של הnext שלו להצביע על מה שמצביע head. ואז מעדכנים את head להצביע על האובייקט החדש new_head.

Get_head – מחזירה את האובייקט שhead מצביע עליו (כתובת בזיכרון).



לרשימה משורשרת יש מבנה שמתאים לחשיבה רקורסיבית על בעיות. אם מורידים את ראש הרשימה נשארים עם רשימה דומה קצרה ב-1 ועם ראש חדש.

```
def get_size(self):
    def recursive_size(list_head):
        if list_head == None:
            return 0
        else:
            return recursive_size(list_head.get_next()) + 1
    return recursive_size(self.__head) # call the inner function
```

עוד דוגמאות (חשובות!) בהרצאה 9.

Recitation 9

טעויות נפוצות שעושים ברשימות מקושרות:

- תוך כדי פעולה מוחקים או משנים את הלינק לאיבר הבא
- לולאה למעבר על איברים ברשימה, כל פעם עושים next. בטעות באוטומט עושים גם next על האיבר האחרון.

מימוש רשימה מקושרת: Node. בונים מחלקה.

```
class Node:
    def __init__(self, data=None, next=None):
        self.__data = data
        self.__next  = next

    def __str__(self):
        return str(self.__data)

    def get_data(self):
        return self.__data

    def get_next(self):
        return self.__next

    def set_data(self,data):
        self.__data = data

    def set_next(self,next):
        self.__next = next
```

ה- next הוא גם מטיפוס node. מצביע על איבר מטיפוס node.

שדות פרטיים. מחוץ למחלקה אי אפשר לגשת אליהם ישירות. לכן צריך לעשות להם getters ו- setters.

אפשר גם לוותר על השדות הפרטיים ולעשות בלי ה- set ו- get. העדפה אישית.

המחלקה linkedlist – מייצגת את הרשימה המקושרת:

```
class LinkedList:
    def __init__(self, head=None):
        self.__head = head

    def get_head(self):
        return self.__head

    def is_empty(self):
        return self.__head == None

    def add(self, new_head):
        new_head.set_next(self.__head)
        self.__head = new_head
```

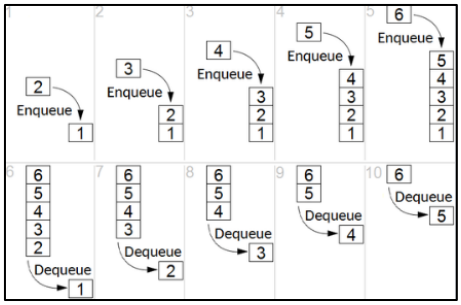
למה צריך את זה אם יש לנו כבר את ה- node? הרבה פעמים באמת לא צריך עוד מחלקה. כאן אפשר ליצור עוד מתודות על הרשימות. *דוגמאות לעוד מתודות החל משקופית 9 בתרגול.*

	רשימה רגילה	רשימה מקושרת
גישה לאיבר ה- i	$O(1)$	$O(n)$ $(O(i))$
Reverse	$O(n)$	$O(n)$
להכניס לראש של רשימה איבר חדש	$O(n)$ (צריך להכניס איבר ולהזיז את כולם הצידה)	$O(1)$
להכניס לאמצע רשימה / להסיר מאמצע הרשימה	$O(n)$ (להזיז חצי מהאיברים)	$O(n)$ בלי רפרנס, $O(1)$ עם רפרנס (כמו רשימה חדשה מהאמצע – רפרנס=head)
להסיר את הראש	$O(n)$	$O(1)$
חיפוש ערך	רשימה לא ממוינת: $O(n)$ רשימה ממוינת: $O(\log(n))$	$O(n)$

Que

תור.

האיבר הראשון שנכנס הוא האיבר הראשון שיוצא.



מפרט (API):

```
class MyQueue:

        def __init__(self):

        def enqueue(self, item):

        def dequeue(self):

        def get_size(self):

        def is_empty(self):
```

Enqueue – הוספת איבר חדש לתור

Dequeue – הסרת איבר מהתור

```
class MyQueue:
    def __init__(self):
        self.__head = None
        self.__tail = None
        self.__size = 0

    def get_size(self):
        return self.__size

    def is_empty(self):
        if self.__size > 0:
            return False
        else:
            return True
```

```
def enqueue(self, item):
    if self.__tail == None:
        self.__head = Node(item)
        self.__tail = self.__head
    else:
        # adding new item to the end of the list
        self.__tail.next = Node(item)
        self.__tail = self.__tail.next
    self.__size += 1
```

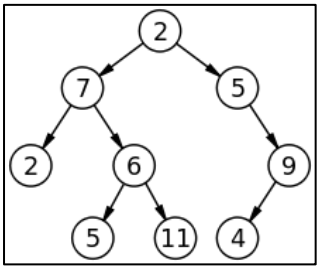
```
def dequeue(self):
    result = self.__head.data
    self.__head = self.__head.next
    if self.__head == None:
        self.__tail = None
    self.__size -= 1
    return result
```

עצים

גרף קשיר ללא מעגלים (דיסקרטית!). גרף מכוון.

יש היררכיה. שורש וחיצים כלפי מטה.

יש מסלול ישיר בין כל שני קודקודים.

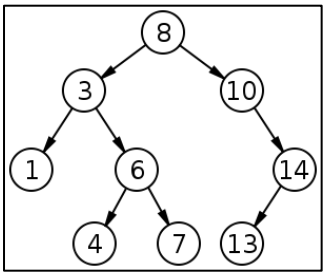


- שורש – ראש העץ, אין חיצים שנכנסים אליו.
- הורה של קודקוד – מי שנמצא מעל הקודקוד ושולח אליו חץ. לכל קודקוד הורה יחיד.
- אחים – קודקודים שיש להם הורה משותף.

- עלים – קודקודים ללא בנים. (בעץ רקורסיה עלים מייצגים את מבנה הבסיס)
- עומק/רמה של קודקוד – המרחק של אותו קודקוד (בצלעות) מהשורש.
- גובה של קודקוד – עומק אבל מלמטה (עלים xבגובה 0)
- גובה העץ – הגובה של השורש – אורך המסלול הארון ביותר בעץ.

עץ בינארי – לכל קודקוד יש לכל היותר שני ילדים.

<u>עץ חיפוש בינארי</u> – עץ בינארי שמקיים: לכל קודקוד בעץ, כל מה שנמצא משמאל קטן יותר ומה שמימין גדול יותר. בנוסף, כל מספר יכול להופיע רק פעם אחת. מימוש לעץ בינארי:




```
root = TreeNode(8, TreeNode(3, TreeNode(1),
TreeNode(6,...,..)), TreeNode(10,...,..))
```

```
class TreeNode:
    def __init__(self, value, left=None, right=None):
        self.__value = value
        self.__left = left
        self.__right = right
```

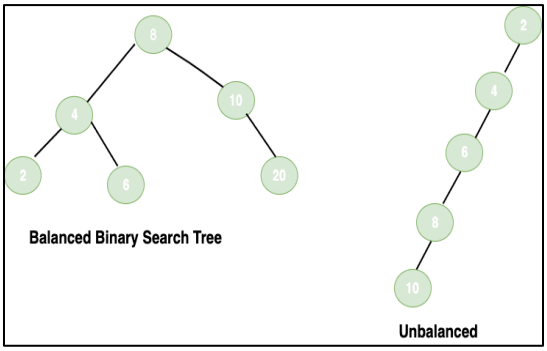
```
class TreeNode:
    ...

    def search(self, value):
        # returns True if value is in the tree
        if self.__value == value:
            return True
        else:
            if value < self.__value:
                if self.__left:
                    return self.__left.search(value)
                else:
                    return False
            else:
                if self.__right:
                    return self.__right.search(value)
                else:
                    return False
```

זמן הריצה של החיפוש:

- המקרה הכי טוב – האיבר נמצא בשורש. $O(1)$.
- המקרה הכי גרוע – צריך לעבור את כל הגובה של העץ. אז O של גובה העץ. $O(n)$. (טעות נפוצה – הגובה הוא $\log(n)$, זה לא נכון כי העץ יכול להיות גם טור ארוך של איברים שכל אחד יותר קטן מהשני – עץ לא מאוזן).

עץ מאוזן – הגובה הוא $\log(n)$ ואז באמת החיפוש לוקח $O(\log(n))$.



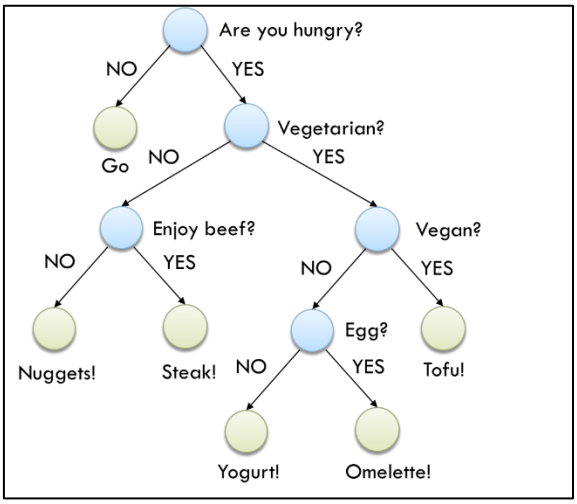

```
def insert(self, item):
    if self.__value == item:
        #we do nothing because the item is already here
        return
    else:
        if item < self.__value:
            if self.__left:
                self.__left.insert(item)
            else:
                self.__left = TreeNode(item)
        else:
            if self.__right:
                self.__right.insert(item)
            else:
                self.__right = TreeNode(item)
```

נלך ימינה ושמאלה עד שנגיע למקום הנכון שרוצים להכניס את האיבר (בתור עלה).

<u>Binary Decision Tree – עץ החלטה</u>

אומרים שעץ הוא **מלא** אם לכל קודקוד יש שני בנים או שהוא עלה.

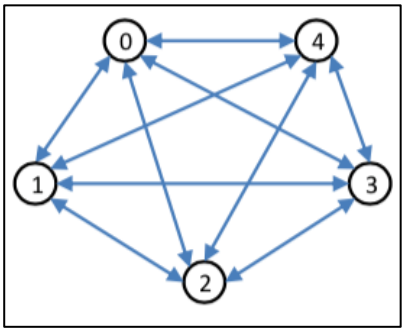
עץ החלטה - אוסף שאלות שבעקבות התשובות יש לנו החלטה.



Lecture 10: Linked data structures: Trees, Graphs

בסרטון הראשון – דוגמאות לגרפים ומימושים. לראות כדי להבין.

Complexity of finding simple maximal paths from a node has become related to $n!$ but also depends on operations in the algorithm at each step (e.g., testing list membership, or set membership)...



עוד מימוש לעץ:

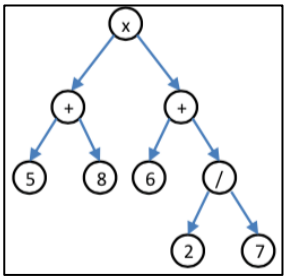
```python
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.children = set()

    def add_child(self, child):
        self.children.add(child)

    def size(self):
        result = 1
        for child in self.children:
            result += child.size()
        return result
```

השיטה size : בלולאת הפור יש זימן רקורסיבי של השיטה עבור כל ילד של הקודקוד הנוכחי אבל גם עבור כל ילד עוברים רקורסיבית על הילדים שלו.

עץ חישוב:




```python
def compute(root):
    if root.data == "+":
        return compute(root.left) + compute(root.right)
    elif root.data == "*":
        return compute(root.left) * compute(root.right)
    elif root.data == "/":
        return compute(root.left) / compute(root.right)
    elif root.data == "-":
        return compute(root.left) - compute(root.right)
    else:
        return float(root.data)
```

עבור כל ביטוי מחשבים קודם את כל תת העץ השמאלי ואז את תת העץ הימני (רקורסיה).

$(3 * 4) + 7$

```python
expr1 = TreeNode("+", TreeNode("*", TreeNode("3"), TreeNode(4)), TreeNode("7"))
```

הדפסת הביטוי, גם באופן רקורסיבי:

```python
# recursive code
def tree_to_expr(tree_root):
    if tree_root.left:
        return "(" + tree_to_expr(tree_root.left) + str(tree_root.data) + tree_to_expr(tree_root.right) + ")"
    else:
        return str(tree_root.data)

expr1 = TreeNode("+", TreeNode("*", TreeNode("3"), TreeNode(4)), TreeNode("7"))

print(tree_to_expr(expr1))
```

```
((3*4)+7)
```

כתיב פולני:

$$((3 * 4) + 7) \quad \Leftrightarrow \quad + * 3\ 4\ 7$$

$$(3 * 4) + (2 - 5) \quad \Leftrightarrow \quad + * 3\ 4 - 2\ 5 \quad \Leftrightarrow \quad (+(* 3\ 4)(-2\ 5))$$

```python
# recursive code
def tree_to_polish(tree_root):
    result = str(tree_root.data)

    if tree_root.left:
        result +=  " " + tree_to_polish(tree_root.left) + " " + tree_to_polish(tree_root.right)
    return result
```

Pre/In/Post Order

ניתן להגדיר בצורה רקורסיבית את הדרך לבקר את כל הקודקודים של עץ בינארי:

Pre Order	שורש, ואז תתי עצים	למשל בכתיב הפולני
In Order	תת עץ שמאלי, שורש, תת עץ ימני	כתיב רגיל
Post Order	תתי עצים, ואז השורש	כתיב פולני הפוך

```python
def print_preorder(root):
    if root:
        print(root.data, end=" ")
        print_preorder(root.left)
        print_preorder(root.right)
```

```python
def print_inorder(root):
    if root:
        print_inorder(root.left)
        print(root.data, end=" ")
        print_inorder(root.right)
```

```python
def print_postorder(root):
    if root:
        print_postorder(root.left)
        print_postorder(root.right)
        print(root.data, end=" ")
```

פונקציות אנונימיות - Lambda Functions

קורים מצבים שיש לנו הרבה שורות עבור פונקציות פשוטות. אז יש פונקציות אנונימיות במקום. נוחות של כתיבה.

לפונקציה אנונימית אין שם. לכולן קוראים lambda.

יכולה להשתמש במשתנים שקיימים בסביבה שבה היא נמצאת.

```
lambda <vars> : <expression>
```

למשל:

```
lambda x : x ** 2 + 5
```

כתיבת הפונקציה ולידה זימון שלה:

```
(lambda x : x ** 2 + 5)(3) # 3 is the arg that binds to x

(lambda x : x / (lambda y, z : y * z)(2, 3))(1) # 2,3 bind to y, z, and 1 binds to x
```

אפשר כל פעם לשנות את הארגומנטים שנותנים לה:

```
def f12(x, y):
    return lambda a, b: a * x + b / y

z = f12(2, 3) # 2,3 go with this z
z(10, 20)
```

```
26.666666666666668
```

```
z(-5, -15)
```

```
-15.0
```

```
lamby = lambda x, y: x ** y - y / 2
lamby(2, 3)
```

העברת פונקציות כארגומנטים לפונקציה אחרת:

```
def f2(a, b, c, d):
    a(b)
    a(c + d)

f2(print, 'hi', 'apple',' banana') # print goes to a
```

```
# list of tuples
records = [('iosi', 27), ('ety', 29), ('moshiko', 25), ('Luisiana', 24)]

# using lambda to mention how to sort (by names length)
sorted(records, key = lambda x: len(x[0]))
```

```
[('ety', 29), ('iosi', 27), ('moshiko', 25), ('Luisiana', 24)]
```

```
# get max value of the age, and use lambda to show where the age is in the tuple
max(records, key = lambda x: x[1])
```

```
('ety', 29)
```

- מיון לפי אורך האיבר הראשון (אורך השם)
-
- מקסימום מבין האיברים השניים (הגיל).

- Note: only one expression in the lambda body; Its value is always returned.
- The lambda expression must fit on one line!

Map

```
map(function, iterable)
```

הפונקציה map מקבלת פונקציה וקונטיינר איטרבילי, מפעילה את הפונקציה על כל אחד מהאובייקטים של הקונטיינר האיטרבילי, ומחזירה את התוצאות באמצעות איטרטור: אחד אחד. מוציאה iterable בעצמה.

```
ns = (1, 3, 2, 4)
res = map(lambda x: x * 1.5 // 2, ns) # result is an iterator
```

```python
def xx(x):
    x**2 # returns None

def xx1(x):
    return x**2

ns = (1, 3, 2, 4)
res0=map(xx,ns)
res1=map(xx1,ns)
print(list(res0))
print(list(res1))
res = map(lambda x: x * 1.5 // 2, ns) # result is an iterator
print(list(res))
```

```
[None, None, None, None]
[1, 9, 4, 16]
[0.0, 2.0, 1.0, 3.0]
```

יכולה לקבל גם יותר מ- iterable אחד בתור אינפוט. שני ארגומטים לפונקציה, אחד מאיטרבל1 ואחד מאיטרבל2.

map(func, iter1, iter2)

Reduce

```
reduce(function, sequence)
```

functools.reduce(f, [1,2,3,4]) == f(f(f(1,2),3),4)

הפונקציה reduce מקבלת פונקציה וסדרה איטרבילית ומפעילה את הפונקציה על איברי הסדרה. כאן כל תוצאה של הפעלת הפונקציה מצטרפת כאופרנד להפעלה הבאה, כך שבסוף ההפעלה מתקבלת תוצאה אחת.

```python
from functools import reduce
reduce(lambda x, y: x + y, (1, 2, 7, 6))
```

$\Longrightarrow \quad (((1+2)+7)+6)$

אם מקבלת ארגומנט שלישי הוא ה"מאתחל". ערך דיפולטיבי.

```
In[3]: from functools import reduce
In[4]: reduce((lambda x, y: x + y), [10,20,30], 1)
Out[4]: 61
```

דוגמה – החזרת מילים ייחודיות:

```python
from functools import reduce

def unique_words(filename):
    with open(filename) as f:
        mapper = map(lambda x: set(x.split()), f)
        return reduce(lambda x,y: x.union(y), mapper)
```

למדא 1: מפרידה את כל המילים

למדא 2: סט של כל המילים (הפונקציה כותבת רק פעם אחת מילים שמופיעות יותר פעמים).

ה- map מוציאה את כל אחת מהמילים באיטרטור.

ה- reduce מחברת את כל הסטים ומחזירה סט אחד.

Filter

```
filter(function, iterable)
```

בונה איטרטור מהאלמנטים של הקונטיינר האיטרבילי עבורם הפונקציה מחזירה ערך אמת.

מכניס ל- iterable רק אלמנטים שהוחזר עליהם true.

```
fibos = [0, 1, 1, 2, 3, 5, 8, 13, 21]
odd_fibos = list(filter(lambda x: x % 2 == 1, fibos))
print(odd_fibos)

even_fibos = list(filter(lambda x: x % 2 == 0, fibos))
print(even_fibos)
```

```
[1, 1, 3, 5, 13, 21]
[0, 2, 8]
```

דוגמה – משפט אינפורמטיבי רק אם יש לו סף מסוים של מילים ייחודיות:

```
from functools import reduce

def unique_informative_words(filename, minimum):
  with open(filename) as f:
    mapper = map(lambda x: set(x.split()), f)
    filtered = filter(lambda x: len(x) >= minimum, mapper)
    return reduce(lambda x,y: x.union(y), filtered)
```

Recitation 10

Exceptions

בפייתון exceptions הם אובייקטים.

המתודה `__getitem__(idx)` מאפשרת להשתמש באינדקסים ברשימות מקושרות ממש כמו ברשימות:

זה המימוש ביחד עם תנאים – למשל אם האינדקס שלילי או גדול מידי.

```
def __getitem__(self, index):
    if index < 0:
        return None
    counter = 0
    node = self.__head
    while counter < index and node:
        node = node.get_next()
        counter += 1
    if not node:
        return None
    return node.get_data()
```

אבל עדיין לא טיפלנו בכל המקרים. יכול להיות שבאחד הnodes יהיה לי ערך None אבל הוא לא האחרון.

עוד דרך בעזרת exceptions – חריגות:

מחליטים בעצמנו על התנהגויות שגויות.

```
def __getitem__(self, index):
    if index < 0 or index >= self.__len:
        raise Exception
    counter = 0
    node = self.__head
    while counter < index:
        node = node.get_next()
        counter += 1
    return node.get_data()
```

אם המקרים האלה קורים: תזרוק שגיאה.

ואז כשנקרא למתודה נשתמש ב:

```
lst = LinkedList()
for i in range(4):
    lst += Node(i)

try:
    print(lst[5])
except:
    print("Cannot access!")
```

אז באיזה פקודות משתמשים:

- `raise Exception`
- `try`
- `except`
- `else`
- `finally`

```
try:

    # do something here  <-- If something goes wrong here (an exception is raised)

except:

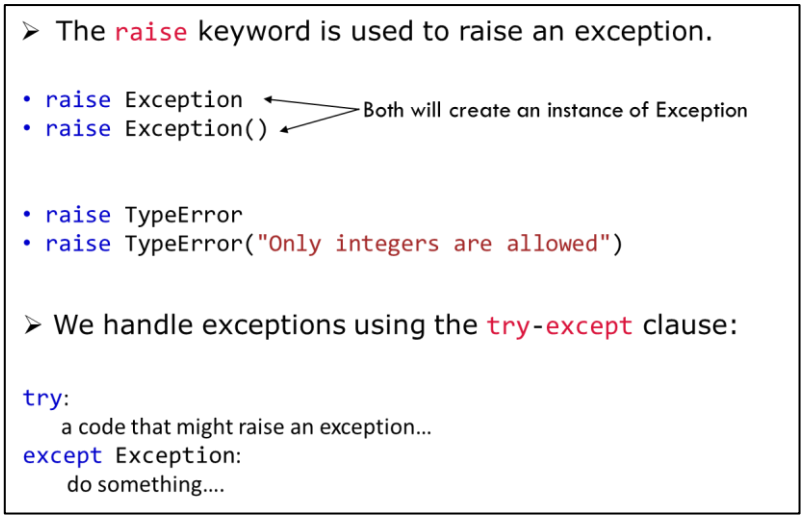
    # handle problems here  <-- we handle it here
```

חריגות – **exceptions** – המכניזם ליצור שגיאות מיוחדות ולטפל בקוד ברגע שנזרקה שגיאה ולשנות את ההתנהגות שלו. במטרה היא לתפוס כמה שיותר שגיאות:

```
def divide(num1, num2):
    try:
        return num1 / num2
    except ZeroDivisionError:
        return "division by zero!"
    except TypeError:
        return "not numbers!"
```

מנסים את ה- exception ואז יש חריגה הוא שם אותה בתוך משתנה בשם e ומדפיס אותה.

```
for i in range(3):
    try:
        print(func1(i))
    except Exception as e:
        print(e)
```



אפשר להשתמש גם ב- **else** ו- **finally**:

Try, אם קרתה שגיאה אז ברגיל except ואם לא קרתה שגיאה אז else. כי אם קרתה שגיאה והוא מחזיר אותה הוא ממשיך בקוד. ולפעמים אנחנו לא רוצים שהוא ימשיך אם קרתה שגיאה. אז רק אם לא – else – תעשה משהו.

ננסה לשים את הדברים ב- try כמה שיותר מצומצמים כדי לצמצם לעצמנו מה השגיאה בדיוק.

Try, אם קרתה שגיאה אז ברגיל except ואם לא קרתה שגיאה אז finally. קורה לא משנה מה – גם אם הייתה וגם אם לא הייתה שגיאה. זה רלוונטי בעיקר כשיש return בקוד. אם ניסיתי לעשות return בלשהו בtry והוא הצליח או לא הצליח או שעשיתי return ב- except מה שבסוף יוחזר זה ה- finally. אם היינו ממשיכים פשוט בקוד בלי לכתוב אותו היה מוחזר פשוט מה שבtry או ב- except והקוד שבהמשך לא היה קורה.

בדר"כ משתמשים ב- finally כדי לעשות cleanup לקוד, ובשביל קריאות לקוד. למשל, כשמתעסקים עם קבצים, לא משנה מה הצלחתי לעשות לקובץ בין אם הצלחתי ובין אם לא אני ארצה בסוף לסגור את הקובץ, תמיד.

Dos and Don'ts

- If an object's constructor has a problem, the best way to detect it is by raising an ***exception***.
- **NEVER** use the ***exception*** mechanism to do things which have another, more natural code pattern.
- Be careful when using ***exceptions*** Especially when catching general ***Exception()*** objects - **they can mask other problems** in the code
- ננסה לתפוס שגיאות כמה שיותר למעלה שהן קורות. לעשות את הtry except כבר בפונקציה העליונה שבה הם קורים

נוסף

מתודות שמחליפות מתודות ברירת מחדל: __str__ הוא print, __add__ הוא +.

- Note the use of the **__str__** and **__add__** methods:
  - Naming the method **__str__** allows us to use:
    - **print**(node) ←→ print(node**.__str__()**)
    - **print**(list) ←→ print(list.**__str__()**)
  - Naming the method **__add__** allows us to use:
    - list.**__add__**(node) ←→ list **+** node

- (By adding the “return self” line we can also use list **+=** node)

Lecture 11: Exceptions

Exception – התרעה על חריגה.

רשימה של כל החריגות של פייתון:

```
BaseException
 +-- SystemExit
 +-- KeyboardInterrupt
 +-- GeneratorExit
 +-- Exception
      +-- StopIteration
      +-- StopAsyncIteration
      +-- ArithmeticError
      |    +-- FloatingPointError
      |    +-- OverflowError
      |    +-- ZeroDivisionError
      +-- AssertionError
      +-- AttributeError
      +-- BufferError
      +-- EOFError
      +-- ImportError
      |    +-- ModuleNotFoundError
      +-- LookupError
      |    +-- IndexError
      |    +-- KeyError
      +-- MemoryError
      +-- NameError
      |    +-- UnboundLocalError
      +-- OSError
      |    +-- BlockingIOError
      |    +-- ChildProcessError
      |    +-- ConnectionError
      |    |    +-- BrokenPipeError
      |    |    +-- ConnectionAbortedError
      |    |    +-- ConnectionRefusedError
      |    |    +-- ConnectionResetError
      |    +-- FileExistsError
      |    +-- FileNotFoundError
      |    +-- InterruptedError
      |    +-- IsADirectoryError
      |    +-- NotADirectoryError
      |    +-- PermissionError
      |    +-- ProcessLookupError
      |    +-- TimeoutError
      +-- ReferenceError
      +-- RuntimeError
      |    +-- NotImplementedError
      |    +-- RecursionError
      +-- SyntaxError
      |    +-- IndentationError
      |         +-- TabError
      +-- SystemError
      +-- TypeError
      +-- ValueError
      |    +-- UnicodeError
      |         +-- UnicodeDecodeError
      |         +-- UnicodeEncodeError
      |         +-- UnicodeTranslateError
      +-- Warning
           +-- DeprecationWarning
           +-- PendingDeprecationWarning
           +-- RuntimeWarning
           +-- SyntaxWarning
           +-- UserWarning
           +-- FutureWarning
           +-- ImportWarning
           +-- UnicodeWarning
           +-- BytesWarning
           +-- ResourceWarning
```

זהו זה בדיוק מה שלמדנו בתרגול.

Recitation 11

בתרגול הזה חוזרים על נושאים שלמדנו אבל ביותר מסובך(:

Iterable vs. Iterator

Iterable –אובייקט שממומשת אצלו המתודה `iter`.

Iterator – האובייקט שמחליט איך המתקדמים קדימה בלולאה. הוא בעצמו iterable. שומר את המצב הפנימי הנוכחי שלו. מחלקה.

המתודה `next`:

- Update the state to point to the next value.
- Return the current element.
- If there are no more elements to return, raise StopIteration.

המתודה `iter` מאפשר להשתמש ב- for loop.

מחלקה (iterator) שמחקה את `range`:

```
class yrange:
    def __init__(self, n):
        self.i = 0
        self.n = n

    def __iter__(self):
        return self

    def __next__(self):
        if self.i < self.n:
            i = self.i
            self.i += 1
            return i
        else:
            raise StopIteration()
```

`range` ו- `yrange` הם iterable. אפשר לשים אותם ב- for loop.

`range` הוא לא איטרטור אז אי אפשר להשתמש עליו ב- `next` לעומת `yrange` שבנינו כאיטרטור עם המתודה `next`.

- By default, if we reach the end of the iterator and then call next, an exception will be raised.
- Instead of raising an exception, we can define a default value to be returned and then check if it was returned.
- We do that by calling `next(iterator, default)`.

```
lst = [1,2,3]

default = object()

iterator = iter(lst)

cur = next(iterator, default)

while cur is not default:
  print(cur)
  cur = next(iterator, default)
```

גנרטורים

גנרטור הוא איטרטור. פונקציה. משתמשים במילה `yield` במקום `return`. ואז אפשר להשתמש על זה במילה `next`. כל פעם שעושים `next` הפונקציה תמשיך לרוץ מהמקום של ה- `yield` עד המקום הבא של ה- `yield` או עד הסוף.

```
def generate_tree(root):

    if root.left:
        for node in root.left:
            yield node

    yield root

    if root.right:
        for node in root.right:
            yield node
```

`yield from` – בדוגמה הקודמת הגנרטור "חבוי" בתוך הלולאת פור. כאן פשוט החלפנו את הלולאה בשורה אחת. תיקח מתוך הגנרטור של `root.left` במקום לעבור על כל ה- `root.left`.

```
def generate_tree(root):

    if root.left:
        yield from generate_tree(root.left)

    yield root

    if root.right:
        yield from generate_tree(root.right)
```

גנרטורים ואיטרטורים אינסופיים:

- Sometimes we will want to be able to iterate over an infinite number of elements.
- We can use iterators and generators in this case too
- We just need to define the way to get the current element at each step.

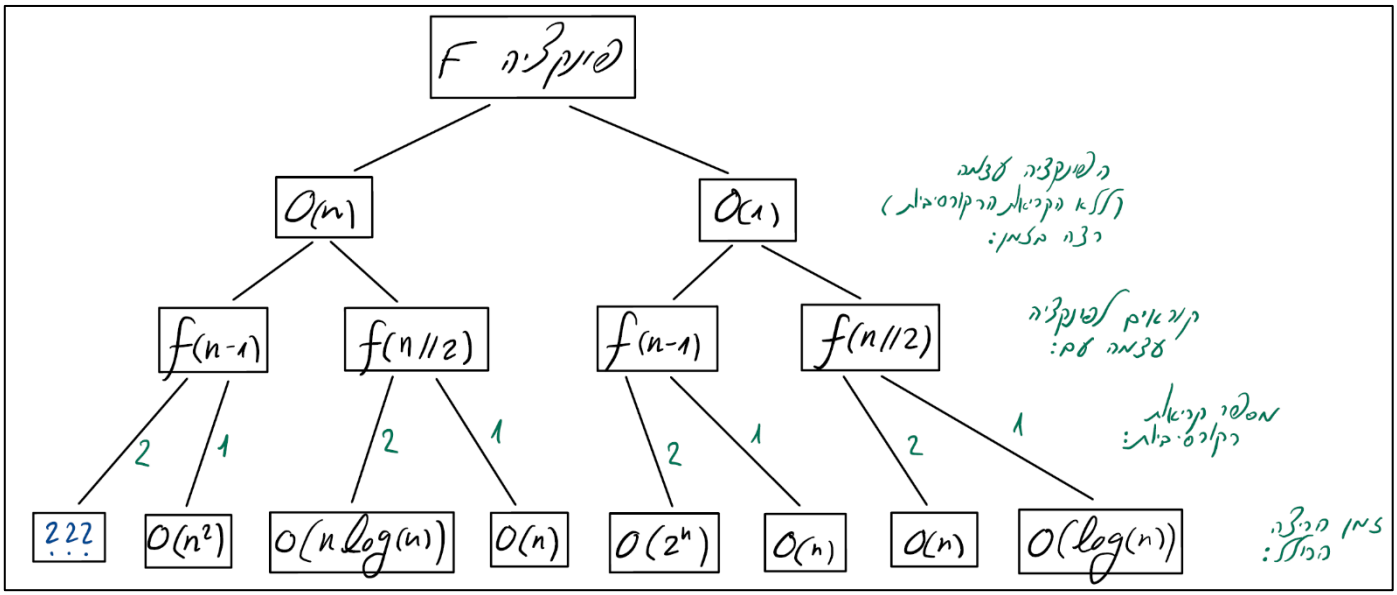
למשל לוח הכפל:

```
def mult_generator(n):
    # Generate all multiples of n.
    cur = 1
    while True:
        yield n * cur
        cur += 1

def table_generator(n):
    # Generate all the numbers in an n by n multiplication table
    for i in range(1,n+1):
        gen = mult_generator(i)
        for _ in range(n):
            yield next(gen)
```

המשך התרגול – map, reduce, filter

זהו נגמר בשם ה' נעשה ונצליח.


פונקציה F
O(n)
O(1)
f(n-1)
f(n//2)
f(n-1)
f(n//2)
2
1
2
1
2
1
2
1
222
O(n²)
O(n log(n))
O(n)
O(2ⁿ)
O(n)
O(n)
O(log(n))